



THE UNIVERSITY
OF QUEENSLAND
AUSTRALIA

CREATE CHANGE

Multimodal RAG

INFS 4205/7205 Advanced Techniques for High Dimensional Data

Yadan Luo

School of EECS, The University of Queensland

Recap: Why RAG Matters

1. Static Knowledge

- LLM's knowledge frozen at the time of training

2. Hallucinations

- LLMs just predict the next word based on all the previous words (-> plausible yet incorrect answers)

3. Cost and Complexity

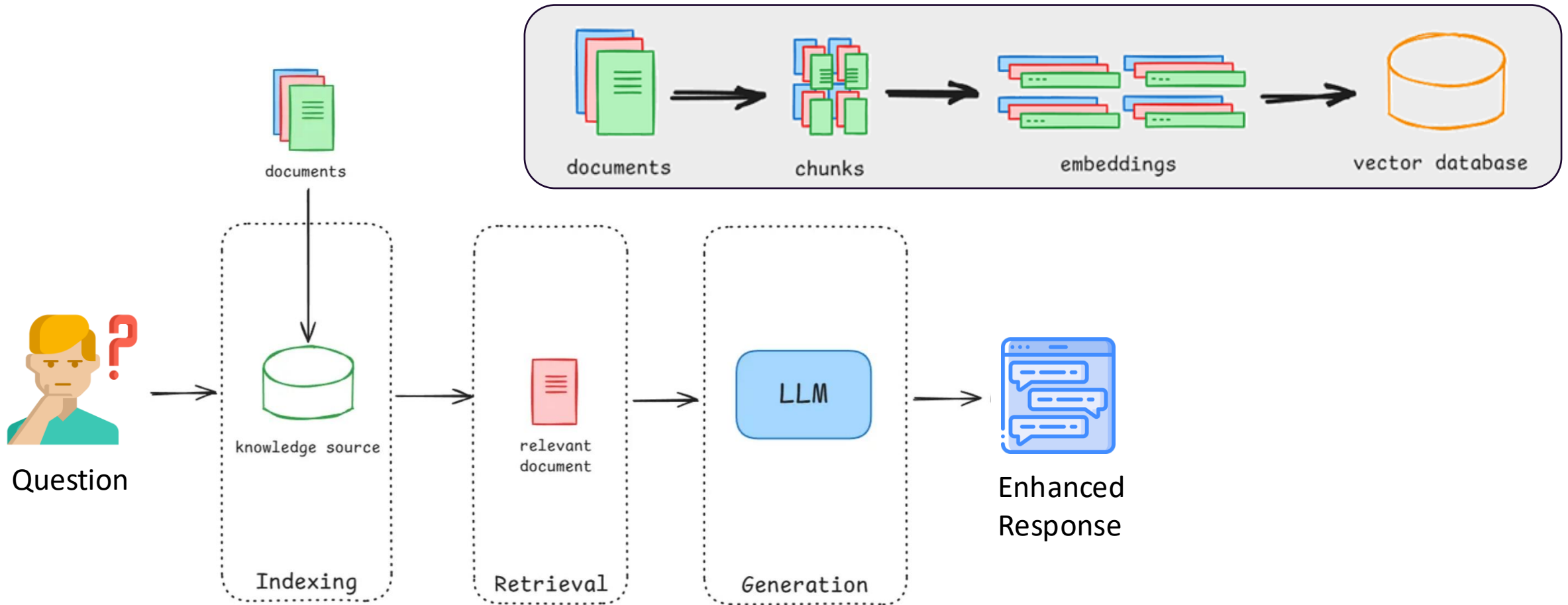
- Retraining requires millions of new examples vs (RAG: in-context learning)

4. Context Window

- Cannot feed all context (context management is 200% more important than models)

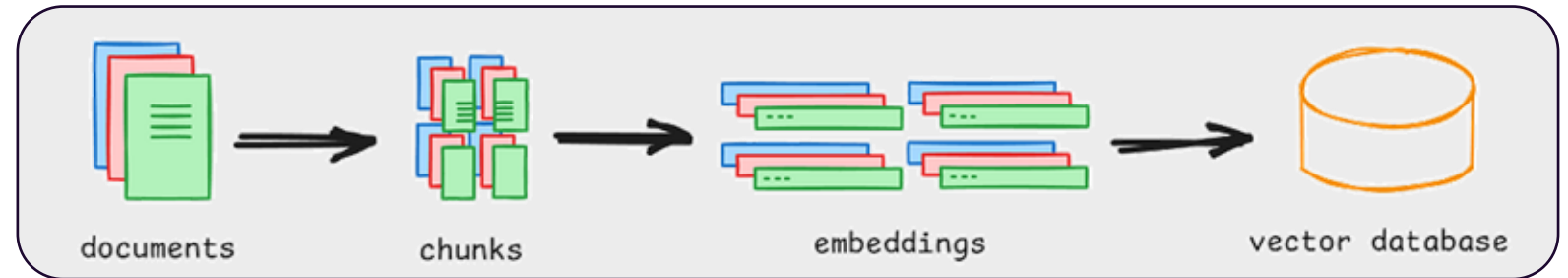
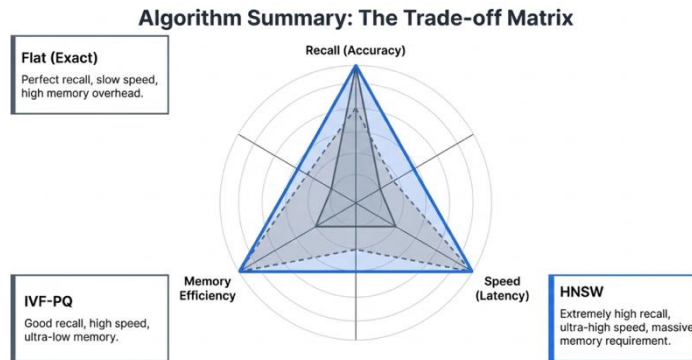
Recap: Text-based RAG

Week 6: High-dim Indexing (**Offline**)



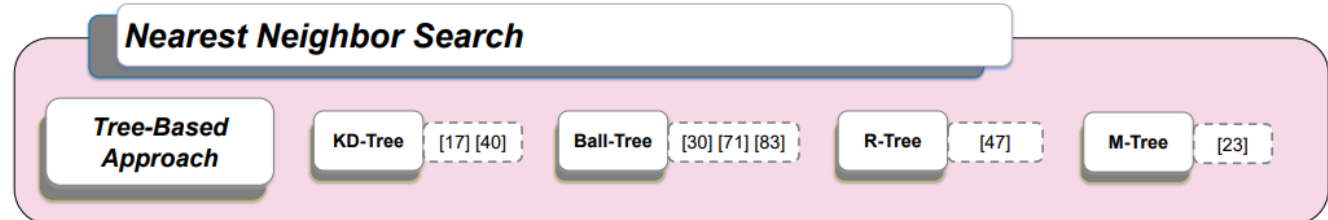
Recap: Text-based RAG

Week 6: High-dim Indexing (Offline)



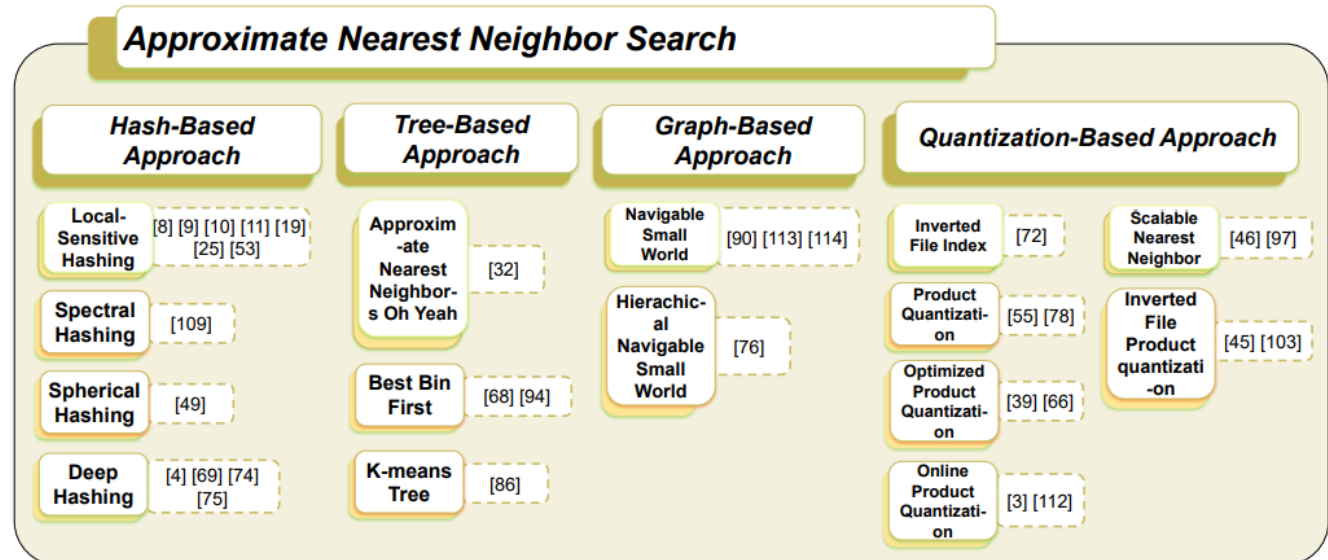
NN Search

- (Exact) more popular for low-dim data, not so applicable to RAG



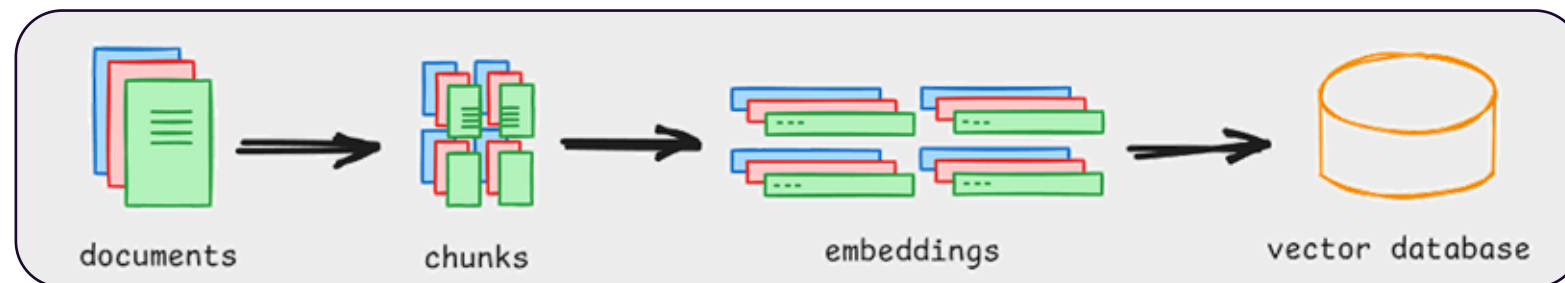
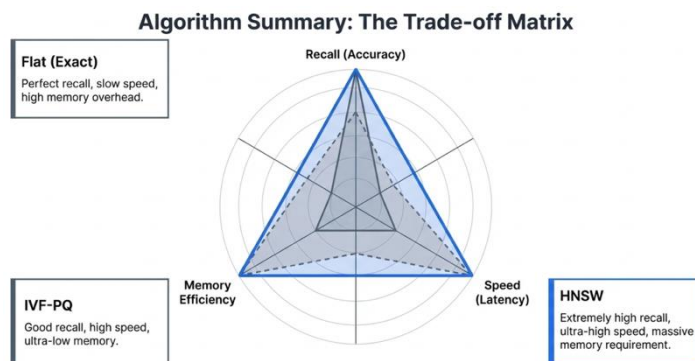
ANN Search

- Quantization (IVF-PQ) is good for large scale
- Hierarchical Navigable Small World (HNSW) is popular, e.g. milvus, FAISS, chroma
- Hash-based doesn't perform well



Recap: Text-based RAG

Week 6: High-dim Indexing (Offline)



NN Search

- (Exact) more popular for low-dim data, not so applicable to RAG

ANN Search

- Quantization (IVF-PQ) is good for large scale
- Hierarchical Navigable Small World (HNSW) is popular, e.g. milvus, FAISS, chroma
- Hash-based doesn't perform as well

	<u>Metrics / Examples (Week 9 Evaluation)</u>
Accuracy	Recall@k, Mean Average Precision, Normalized Discounted Cumulative Gain (NDCG), Mean Reciprocal Rank
Efficiency	Latency (p95/p99), Throughput (QPS)
Scalability	No. of vectors, build time
Resource use	Memory footprint, index size
Flexibility	Cosine, L2, IP, multimodal support

Recap: Text-based RAG

Week 6: High-dim Indexing (Offline)

retro & new twist



TODAY'S RECIPE

Spinach-Cheese Balls

Prep Time: 10 Minutes
Start to Finish: 25 Minutes
Servings: 30 appetizers

RETRO RECIPE

Sausage-Cheese Balls

Prep Time: 30 Minutes
Start to Finish: 55 Minutes
Servings: About 100 appetizers

3 cups Original Bisquick® mix
1 lb bulk pork sausage
4 cups shredded Cheddar cheese (1 lb)
1/2 cup grated Parmesan cheese
1/2 cup milk
1/2 teaspoon dried rosemary leaves, crushed
1/2 teaspoon chopped fresh parsley or
1/2 teaspoon parsley flakes

1 Heat oven to 350°F. Spray 15x10x1-inch pan with cooking spray.
2 In large bowl, stir all ingredients until well mixed. Shape mixture into 1-inch balls; place in pan.
3 Bake uncovered 20 to 25 minutes or until no longer pink in center. Immediately remove from pan. Serve warm.

PER APPETIZER: Calories 46; Total Fat 2.5g (Saturated Fat 1g); Sodium 130mg; Total Carbohydrate 3g (Dietary Fiber 0g); Protein 2g

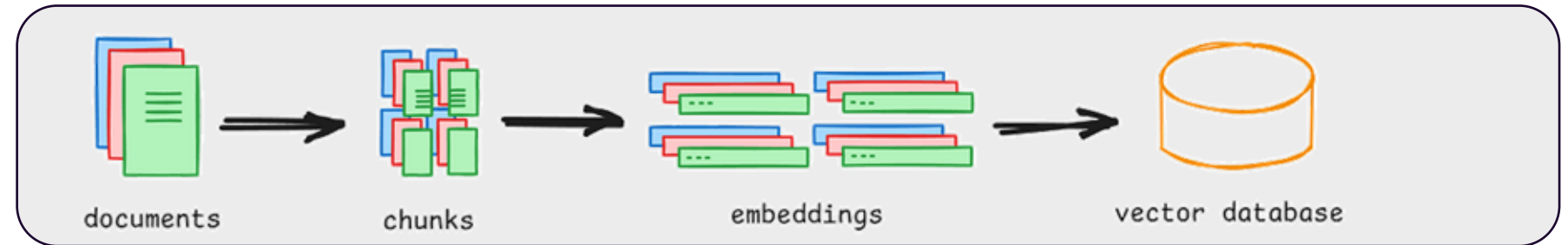
HOW-TO
To quickly thaw spinach, cut small slit in center of pouch; microwave on High 2 to 3 minutes or until thawed. Remove spinach from pouch; squeeze dry with paper towels.

© Betty Crocker All-Time Best Recipes

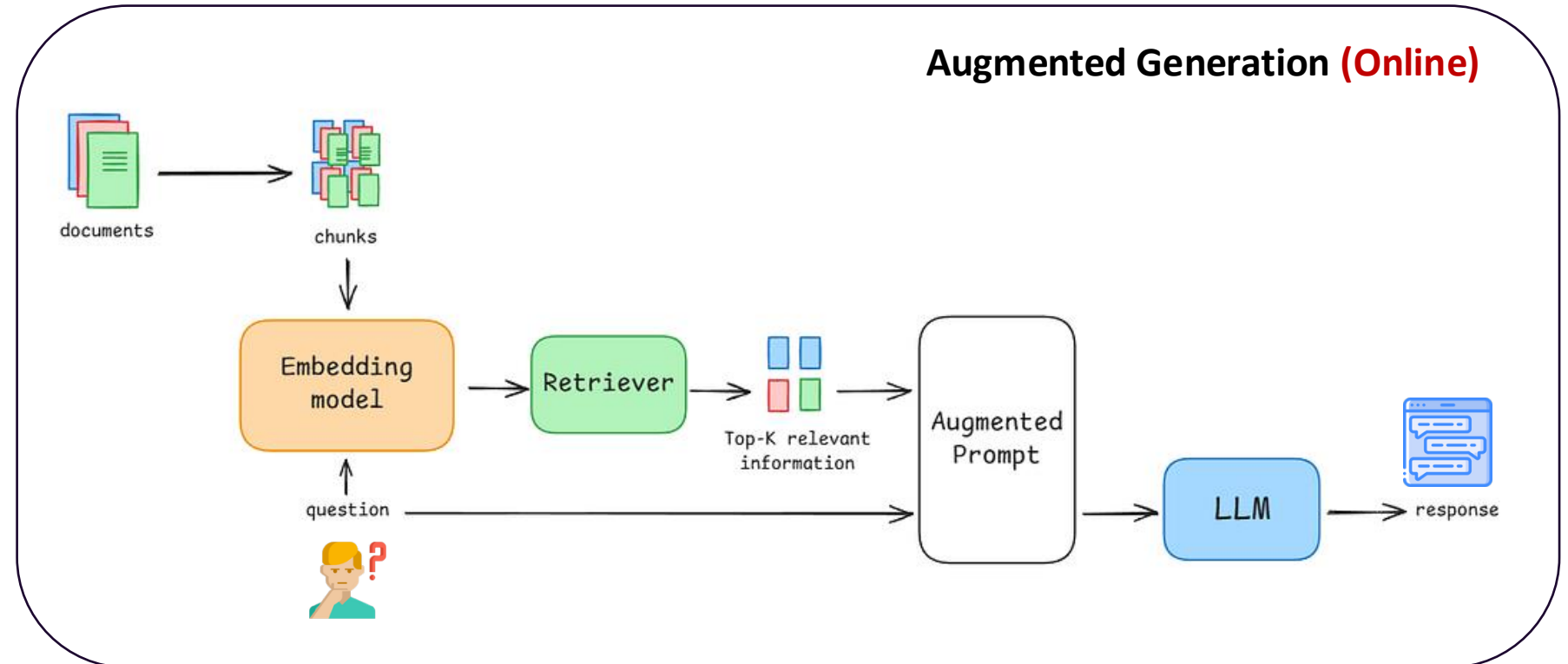
pymupdf4llm

Text Chunks

- > Chunk 1 · page 1 · 97 chars
- > Chunk 2 · page 2 · 376 chars
- > Chunk 3 · page 3 · 1172 chars
- > Chunk 4 · page 4 · 1953 chars



Augmented Generation (Online)



Recap: Text-based RAG

Week 6: High-dim Indexing (Offline)

retro & new twist



TODAY'S RECIPE

Spinach-Cheese Balls

Prep Time: 10 Minutes
Start to Finish: 25 Minutes
Servings: 30 appetizers

RETRO RECIPE

Sausage-Cheese Balls

Prep Time: 30 Minutes
Start to Finish: 55 Minutes
Servings: About 100 appetizers

3 cups Original Bisquick® mix
1 lb bulk pork sausage
4 cups shredded Cheddar cheese (1 lb)
1/2 cup grated Parmesan cheese
1/2 cup milk
1/2 teaspoon dried rosemary leaves, crushed
1/2 teaspoon chopped fresh parsley or
1/2 teaspoon parsley flakes

1 Heat oven to 350°F. Spray 15x10x1-inch pan with cooking spray.
2 In large bowl, stir all ingredients until well mixed. Shape mixture into 1-inch balls; place in pan.
3 Bake uncovered 20 to 25 minutes or until no longer pink in center. Immediately remove from pan. Serve warm.

PER APPETIZER: Calories 46; Total Fat 2.5g (Saturated Fat 1g); Sodium 130mg; Total Carbohydrate 3g (Dietary Fiber 0g); Protein 2g

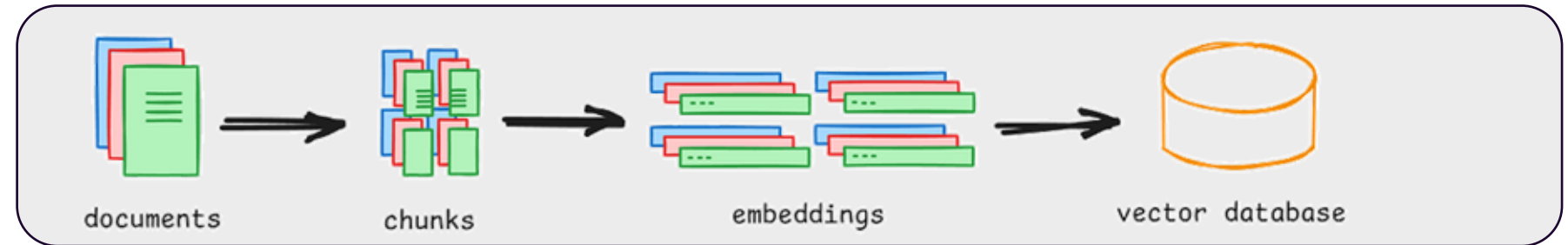
HOW-TO
To quickly thaw spinach, cut small slit in center of pouch; microwave on High 2 to 3 minutes or until thawed. Remove spinach from pouch; squeeze dry with paper towels.

© Betty Crocker All-Time Best Recipes

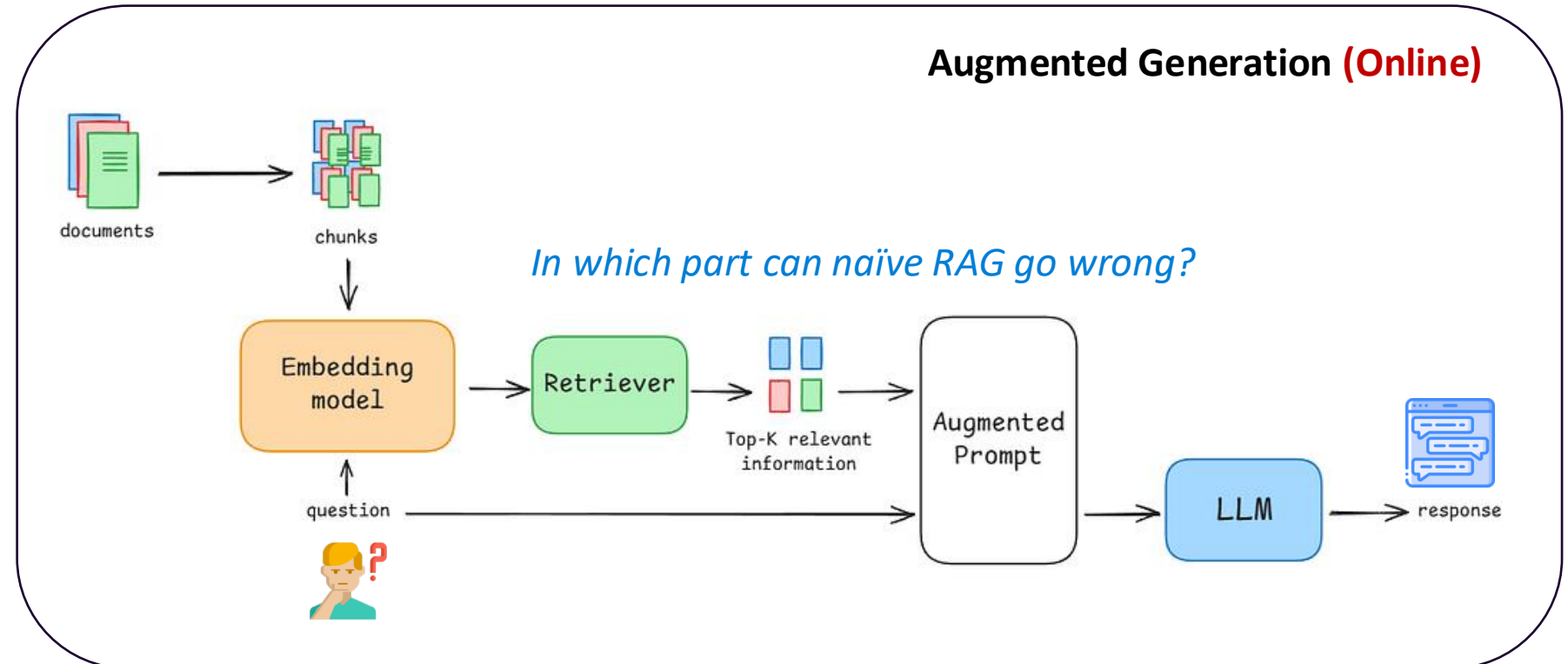
pymupdf4llm

Text Chunks

- > Chunk 1 · page 1 · 97 chars
- > Chunk 2 · page 2 · 376 chars
- > Chunk 3 · page 3 · 1172 chars
- > Chunk 4 · page 4 · 1953 chars

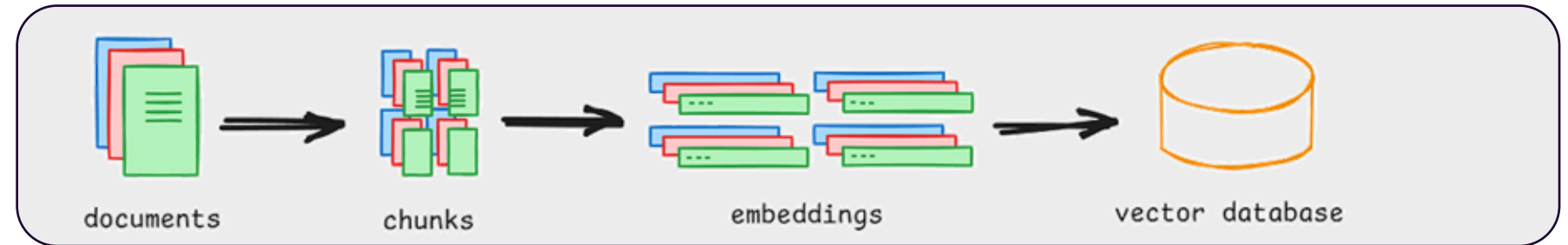


Augmented Generation (Online)

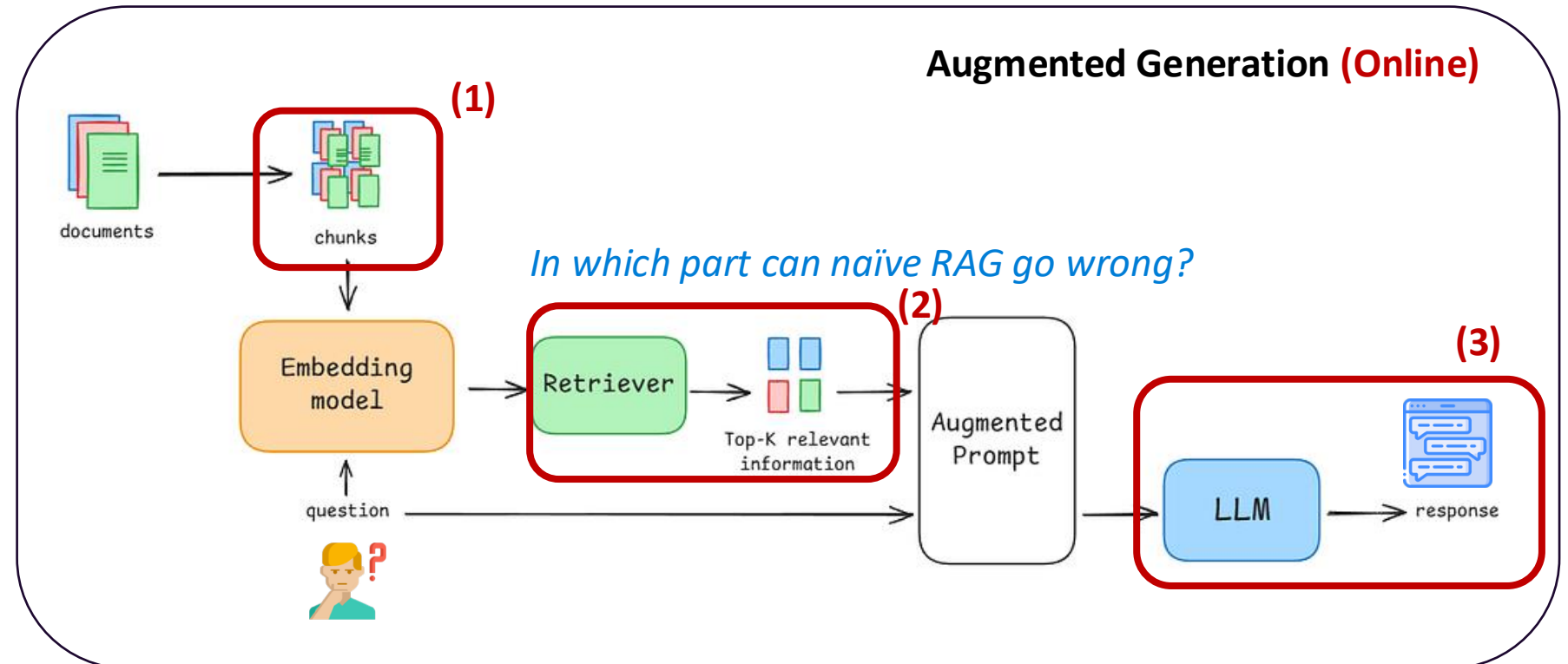


Limitations of Naive RAG

Week 6: High-dim Indexing (Offline)



Augmented Generation (Online)

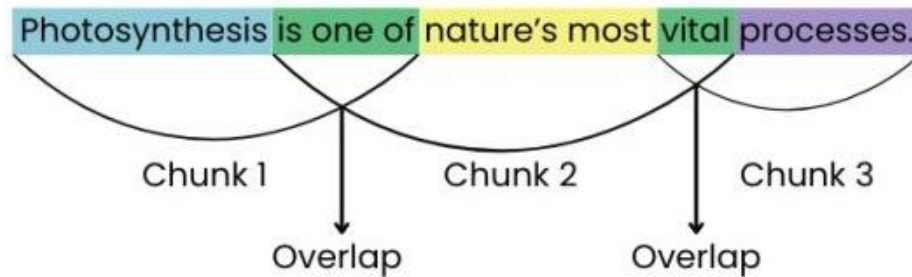


Possible Pitfalls...

1. Broken Chunks
2. Irrelevant Documents
3. Wrong Conclusion
- ...

Chunking - Limitations of Naive RAG

- Chunking is the process of breaking long texts into **manageable pieces**, making it easier for retrieval systems and language models to work effectively.



Fixed-size Chunking

- Straightforward but
- may split ideas mid-sentence

pymupdf4llm



▼ Chunk 17 · page 13 · 765 chars

****3**** In large bowl, beat cream cheese with spoon until smooth. E tablespoons of the Parmesan cheese, the milk, garlic and basil un spinach and artichokes. Spread evenly in baking dish. Sprinkle wit Parmesan cheese.

****4**** Bake uncovered 20 to 25 minutes or until thoroughly heated
****PER SERVING (2 TABLESPOONS DIP):**** Calories 60; Total Fat 4g (140mg; Total Carbohydrate 3g (Dietary Fiber 1g); Protein 3g

WE'VE REPLACED INGREDIENTS WITH GREAT-TASTING ALTERNATI

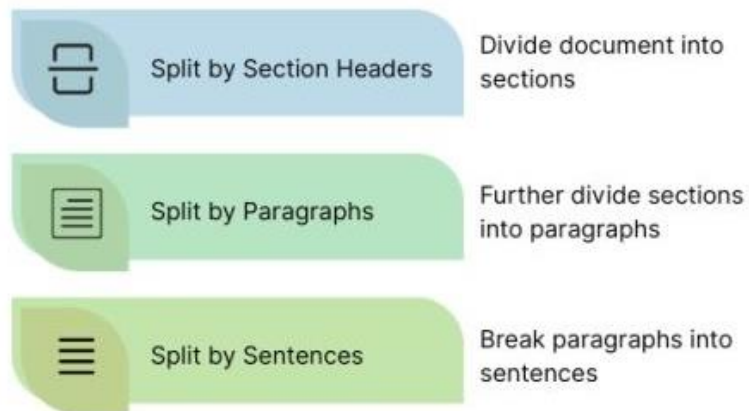
▼ Chunk 18 · page 14 · 1744 chars

retro & new twist

THIS DIP BECAME POPULAR IN THE 1980S, COINCIDING WITH THE MEXICAN FOOD IN AMERICA.

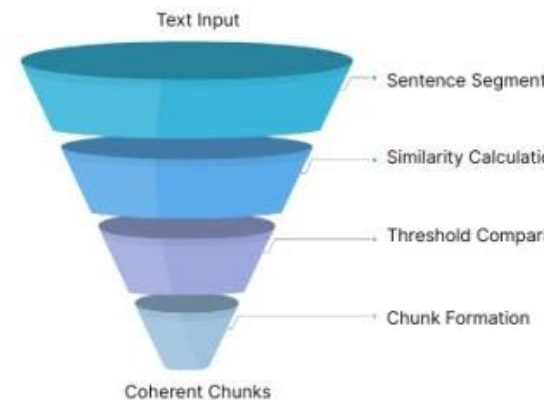
Chunking - Limitations of Naive RAG

- Chunking is the process of breaking long texts into manageable pieces, making it easier for retrieval systems and language models to work effectively.



Recursive Chunking

- More structured and hierarchical
- Start with sections.. If too large then smaller units



Semantic Chunking

- Focus on meaning
- Analyse semantic relationship and determine where to split (new topic)
- Computational expensive 😞

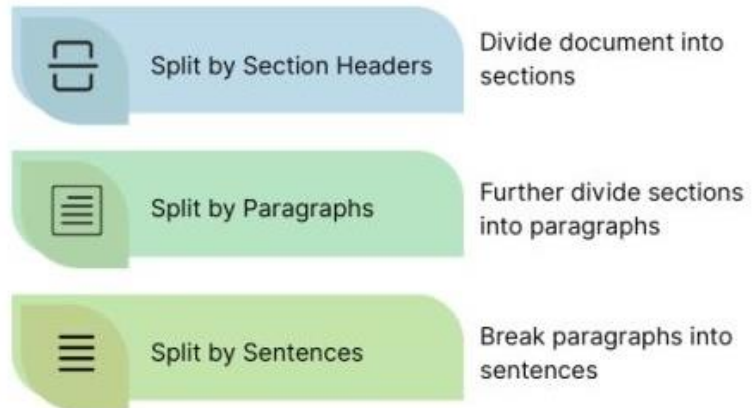


Agentic Chunking

- Let LLMs to decide
- Token expensive 😞

Chunking - Limitations of Naive RAG

- Practical Implementation Tips



Recursive Chunking

- More structured and hierarchical
- Start with sections.. If too large then smaller units

LangChain Docs [Open source](#) [Ask](#)

Deep Agents LangChain LangGraph **Integrations** Learn Reference Contribute

Python

LangChain integrations
All providers

Popular Providers

OpenAI
Anthropic
Google
AWS
NVIDIA
Hugging Face
Microsoft
Ollama
Groq

Integrations by component

Chat models
Tools and toolkits
Middleware
Sandboxes
Checkpointers
Retrievers
Text splitters
Embedding models
Vector stores
Document loaders

- How the text is split: by list of characters.
- How the chunk size is measured: by number of characters.

Below we show example usage.

```
pip install -qU langchain-text-splitters
```

To obtain the string content directly, use `.split_text`.

To create LangChain `Document` objects (e.g., for use in downstream tasks), use `.create_documents`.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

# Load example document
with open("state_of_the_union.txt") as f:
    state_of_the_union = f.read()

text_splitter = RecursiveCharacterTextSplitter(
    # Set a really small chunk size, just to show.
    chunk_size=100,
    chunk_overlap=20,
    length_function=len,
    is_separator_regex=False,
)

texts = text_splitter.create_documents([state_of_the_union])
print(texts[0])
print(texts[1])
```

page_content='Madam Speaker, Madam Vice President, our First Lady and
page_content='of Congress and the Cabinet. Justices of the Supreme Court. My

```
print(text_splitter.split_text(state_of_the_union)[:2])
```

```
['Madam Speaker, Madam Vice President, our First Lady and Second Gentl
```

- Chunk size

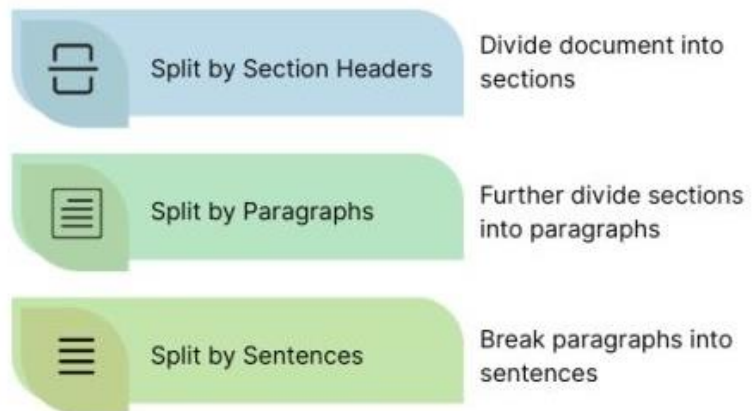
- **Upperbound** (text_embed max tokens, e.g., BERT 512, text-embedding-3, 8192)
- **Recommended** (250-512 tokens for general docs, 500-1000 for legal docs)

- Overlap (10%~20%, tech docs -> 25%)

- How to test?

Chunking - Limitations of Naive RAG

- Practical Implementation Tips



Recursive Chunking

- More structured and hierarchical
- Start with sections.. If too large then smaller units

LangChain Docs [Open source](#) [Ask](#)

Deep Agents LangChain LangGraph **Integrations** Learn Reference Contribute

Python

LangChain integrations
All providers

Popular Providers

OpenAI
Anthropic
Google
AWS
NVIDIA
Hugging Face
Microsoft
Ollama
Groq

Integrations by component

Chat models
Tools and toolkits
Middleware
Sandboxes
Checkpointers
Retrievers
Text splitters
Embedding models
Vector stores
Document loaders

- How the text is split: by list of characters.
- How the chunk size is measured: by number of characters.

Below we show example usage.

```
pip install -qU langchain-text-splitters
```

To obtain the string content directly, use `.split_text`.

To create LangChain `Document` objects (e.g., for use in downstream tasks), use `.create_documents`.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

# Load example document
with open("state_of_the_union.txt") as f:
    state_of_the_union = f.read()

text_splitter = RecursiveCharacterTextSplitter(
    # Set a really small chunk size, just to show.
    chunk_size=100,
    chunk_overlap=20,
    length_function=len,
    is_separator_regex=False,
)

texts = text_splitter.create_documents([state_of_the_union])
print(texts[0])
print(texts[1])
```

page_content='Madam Speaker, Madam Vice President, our First Lady and
page_content='of Congress and the Cabinet. Justices of the Supreme Court. My

```
print(text_splitter.split_text(state_of_the_union)[:2])
```

```
['Madam Speaker, Madam Vice President, our First Lady and Second Gentl
```

- Chunk size

- **Upperbound** (text_embed max tokens, e.g., BERT 512, text-embedding-3, 8192)
- **Recommended** (250-512 tokens for general docs, 500-1000 for legal docs)

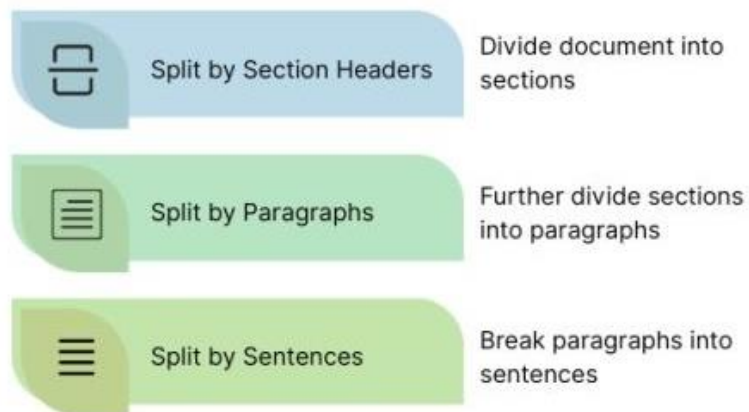
- Overlap (10%~20%, tech docs -> 25%)

- How to test?

Chunking - Limitations of Naive RAG

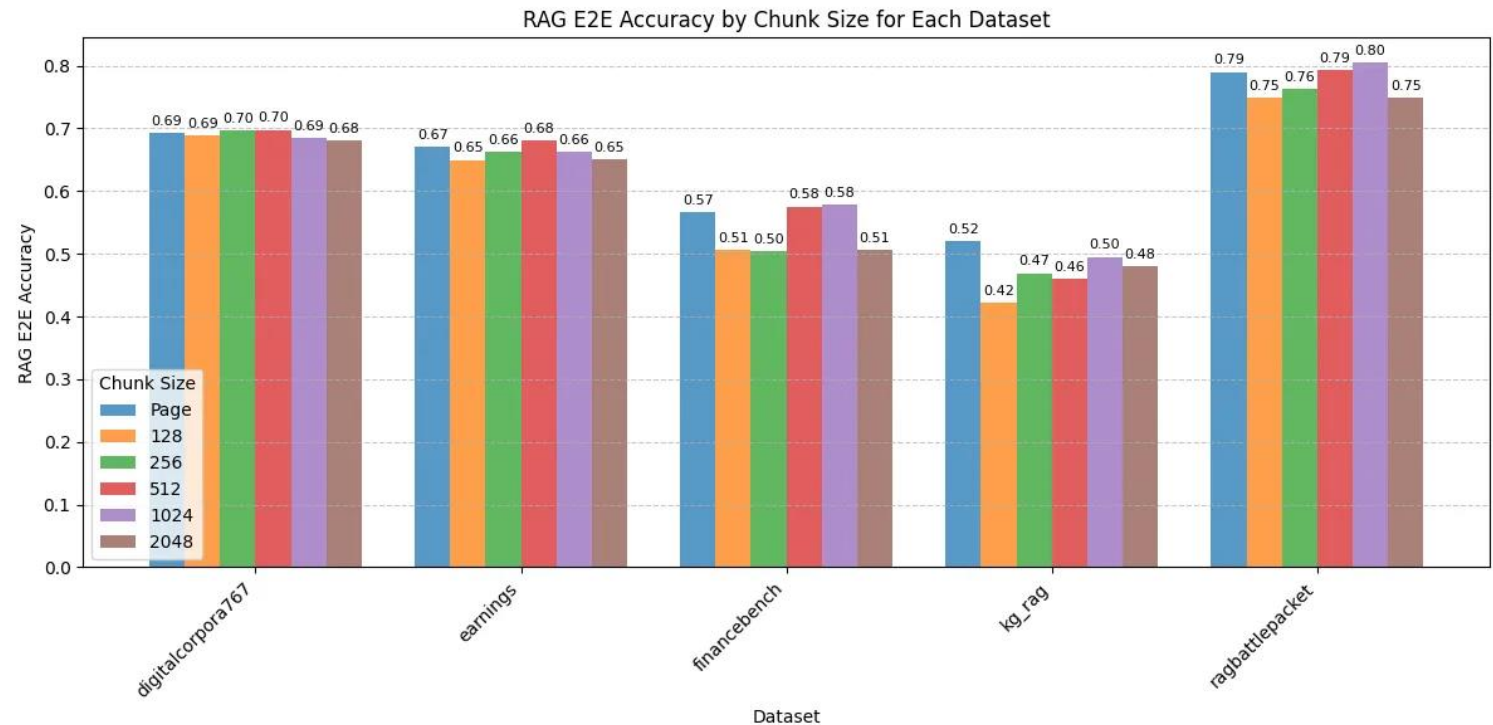
- Practical Implementation Tips

- How to test?



Recursive Chunking

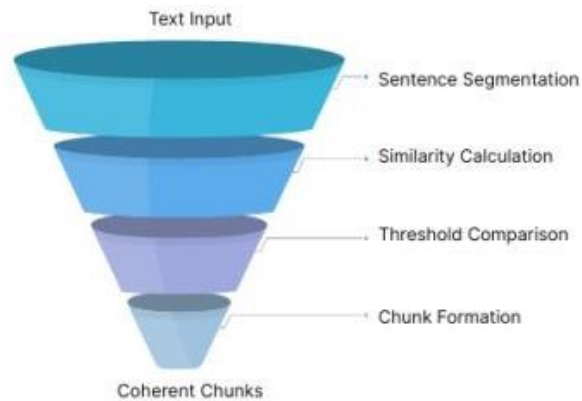
- More structured and hierarchical
- Start with sections.. If too large then smaller units



1. Start with page-level chunking as your baseline.
2. Select 1-2 additional chunking strategies based on your content type.
3. Run a small-scale evaluation on your dataset.
4. Analyze both quantitative metrics and qualitative response quality.
5. Iterate and refine based on results.

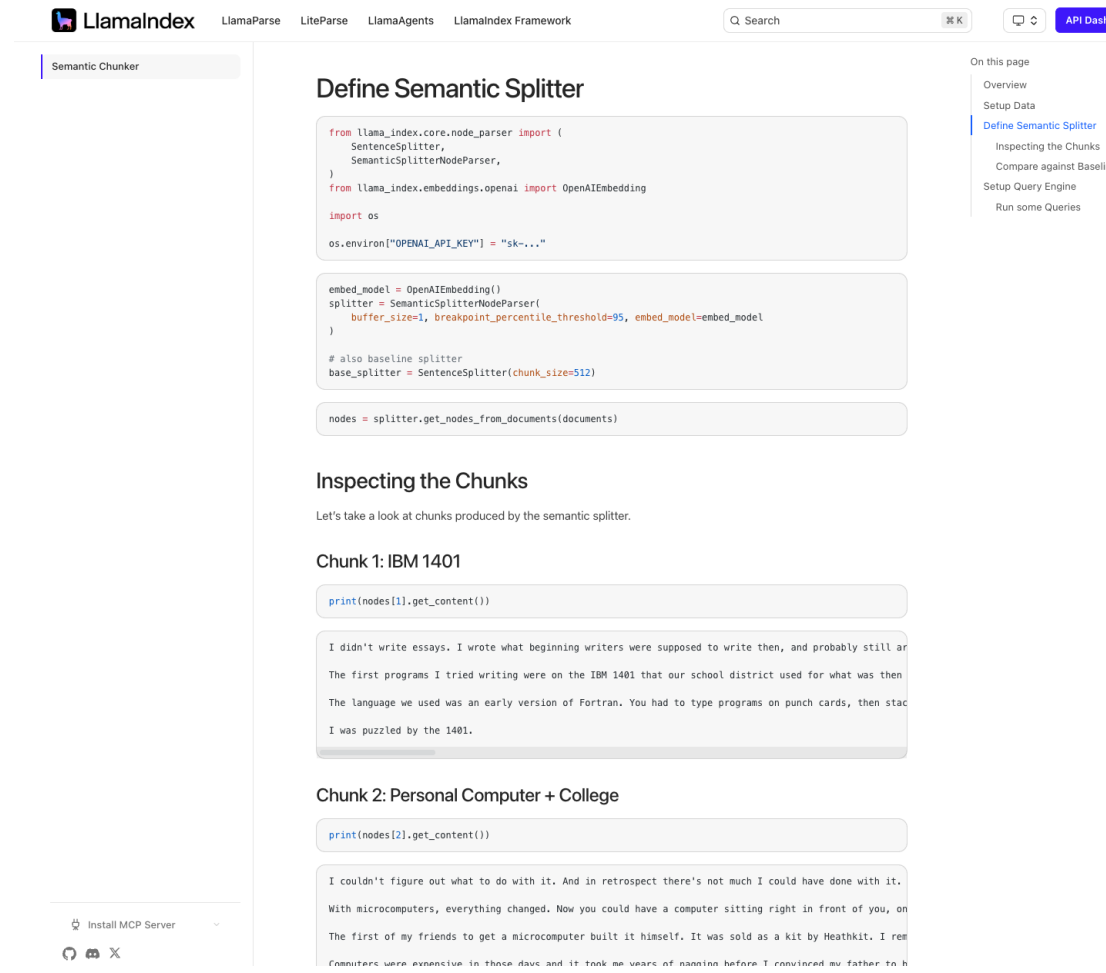
Chunking - Limitations of Naive RAG

- Chunking is the process of breaking long texts into manageable pieces, making it easier for retrieval systems and language models to work effectively.



Semantic Chunking

- Focus on meaning
- Analyse semantic relationship and determine where to split (**new topic**)
- Computational expensive 😞



The screenshot shows the LlamaIndex documentation page for Semantic Chunking. The page title is "Define Semantic Splitter". It includes a code block for setting up the semantic splitter and a section titled "Inspecting the Chunks" which displays two example chunks of text.

Define Semantic Splitter

```
from llama_index.core.node_parser import (
    SentenceSplitter,
    SemanticSplitterNodeParser,
)
from llama_index.embeddings.openai import OpenAIEmbedding
import os

os.environ["OPENAI_API_KEY"] = "sk-..."

embed_model = OpenAIEmbedding()
splitter = SemanticSplitterNodeParser(
    buffer_size=1, breakpoint_percentile_threshold=95, embed_model=embed_model
)

# also baseline splitter
base_splitter = SentenceSplitter(chunk_size=512)

nodes = splitter.get_nodes_from_documents(documents)
```

Inspecting the Chunks

Let's take a look at chunks produced by the semantic splitter.

Chunk 1: IBM 1401

```
print(nodes[1].get_content())
```

I didn't write essays. I wrote what beginning writers were supposed to write then, and probably still are. The first programs I tried writing were on the IBM 1401 that our school district used for what was then the language we used was an early version of Fortran. You had to type programs on punch cards, then stack them. I was puzzled by the 1401.

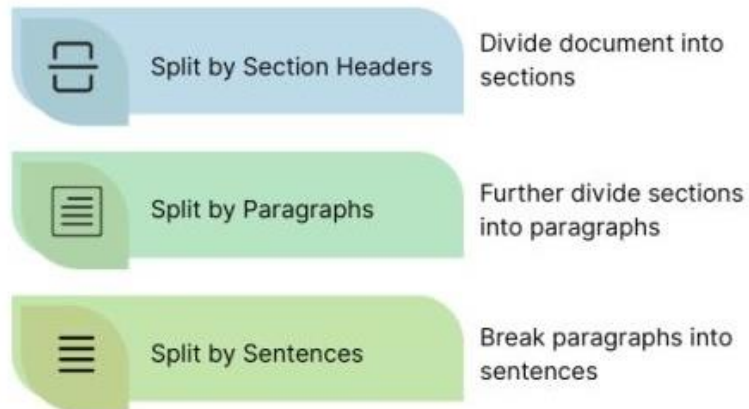
Chunk 2: Personal Computer + College

```
print(nodes[2].get_content())
```

I couldn't figure out what to do with it. And in retrospect there's not much I could have done with it. With microcomputers, everything changed. Now you could have a computer sitting right in front of you, on your desk. The first of my friends to get a microcomputer built it himself. It was sold as a kit by Heathkit. I remember that. Computers were expensive in those days and it took me years of nagging before I convinced my father to buy one.

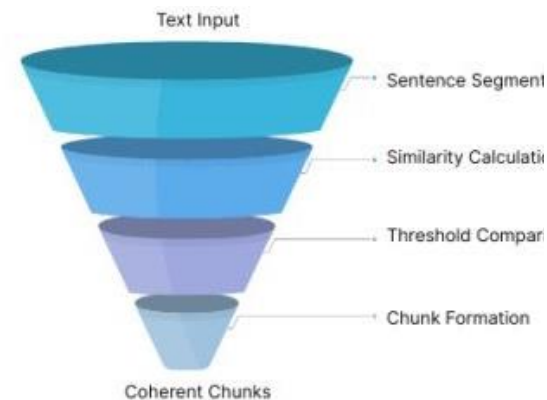
Chunking - Limitations of Naive RAG

- Chunking is the process of breaking long texts into manageable pieces, making it easier for retrieval systems and language models to work effectively.



Recursive Chunking

- More structured and hierarchical
- Start with sections.. If too large then smaller units



Semantic Chunking

- Focus on meaning
- Analyse semantic relationship and determine where to split (new topic)
- Computational expensive 😞



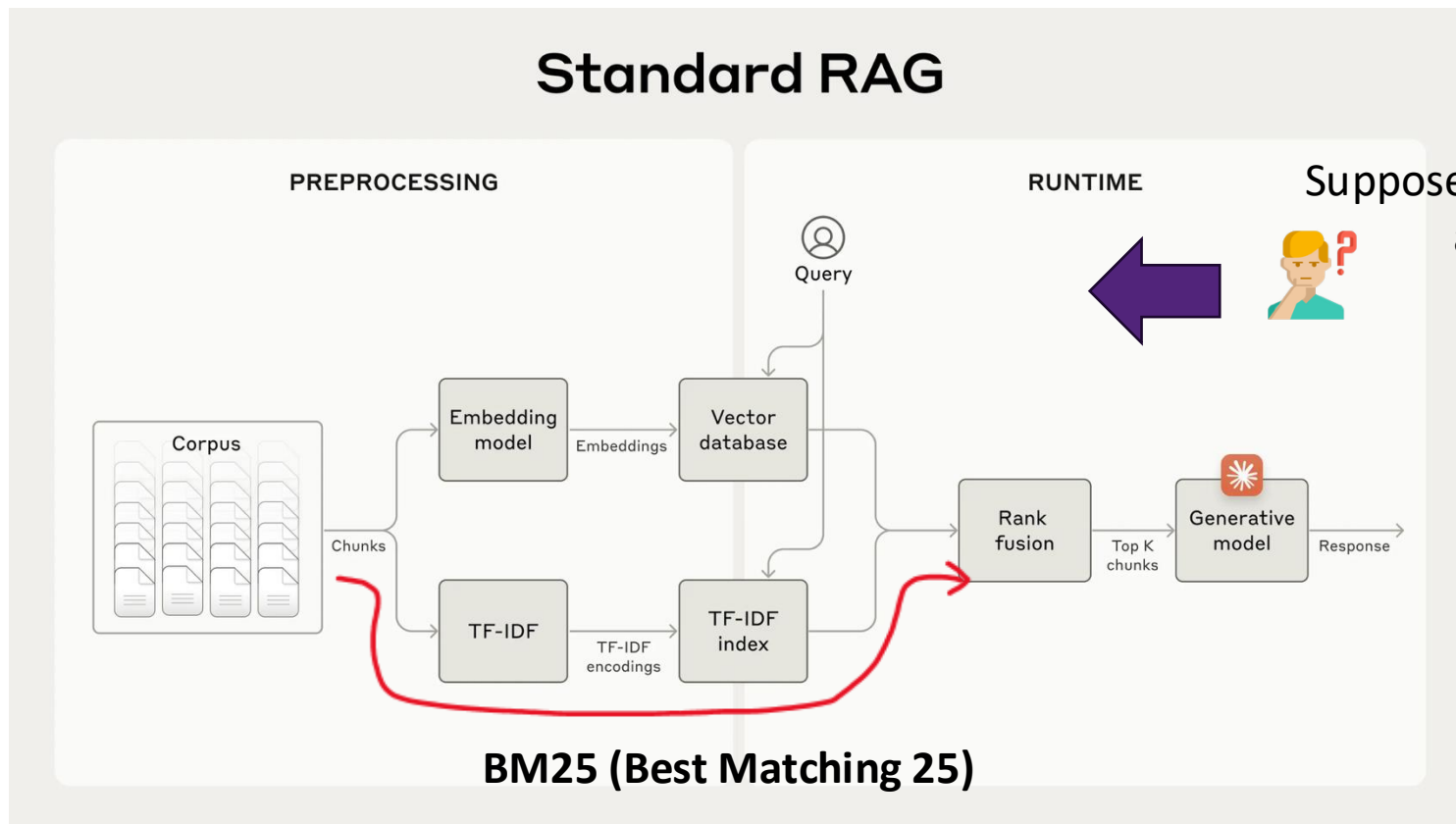
Agentic Chunking

- Let LLMs to decide
- Token expensive 😞

Advanced Chunking

- All chunking methods still inevitably lose critical information...

Contextual Retrieval (Anthropic, 2024)



Embedding model ->

- find content about error codes
- but may miss "TS-999"

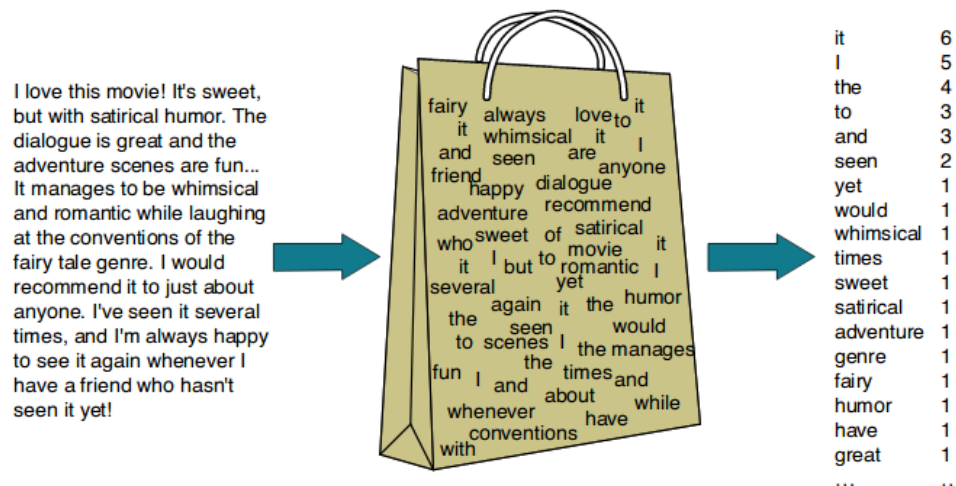
BM25 ->

- Specific text string matching

Advanced Chunking

Sparse Retrieval - Best Matching (BM25)

- is a bag-of-words retrieval function that ranks a set of documents based on the query terms appearing in each document

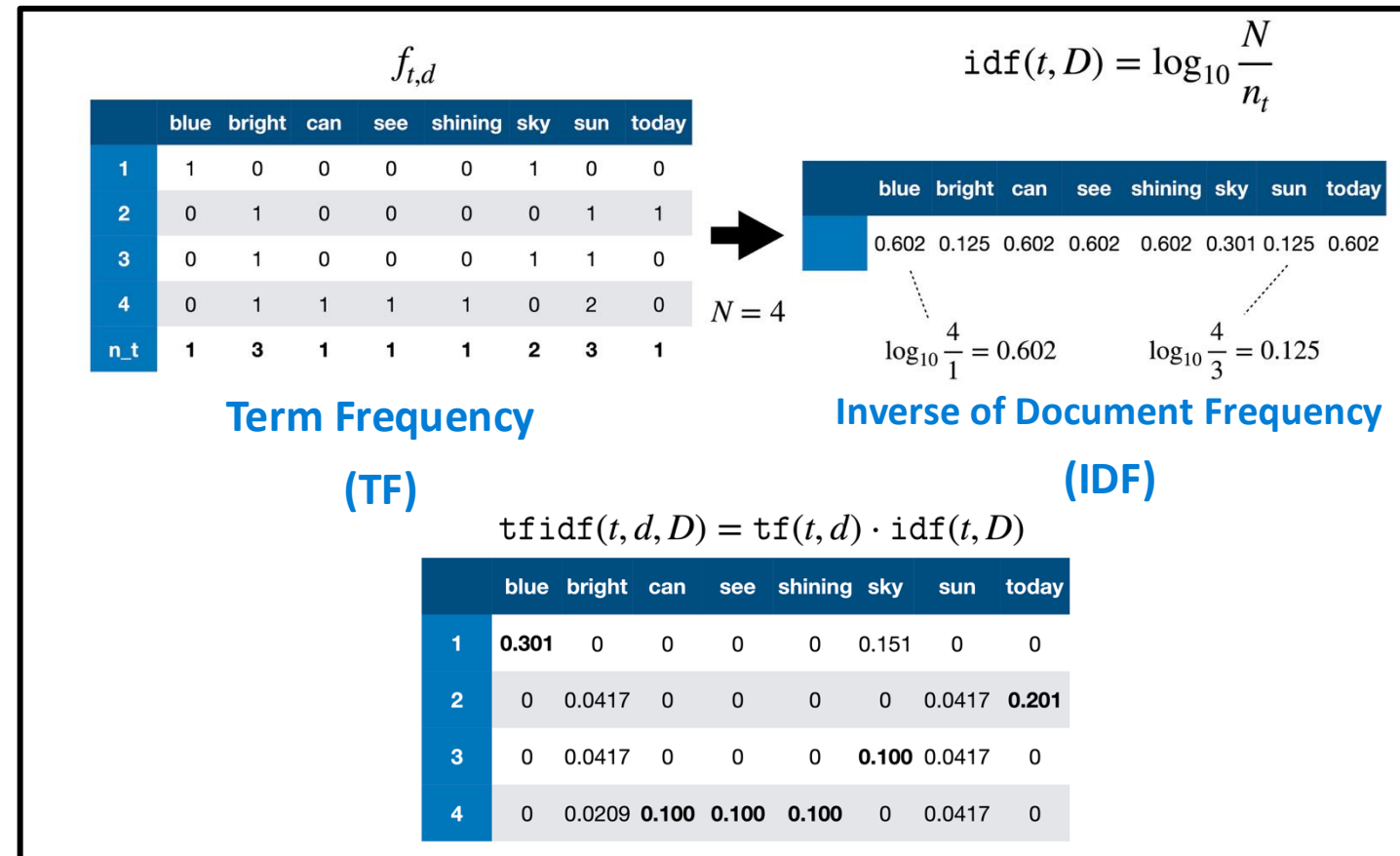


$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

where:

$$\text{TF}(t, d) = \frac{\text{Number of occurrences of } t \text{ in } d}{\text{Total number of terms in } d}$$

$$\text{IDF}(t) = \log \left(\frac{\text{Total number of documents}}{\text{Number of documents containing } t} \right)$$

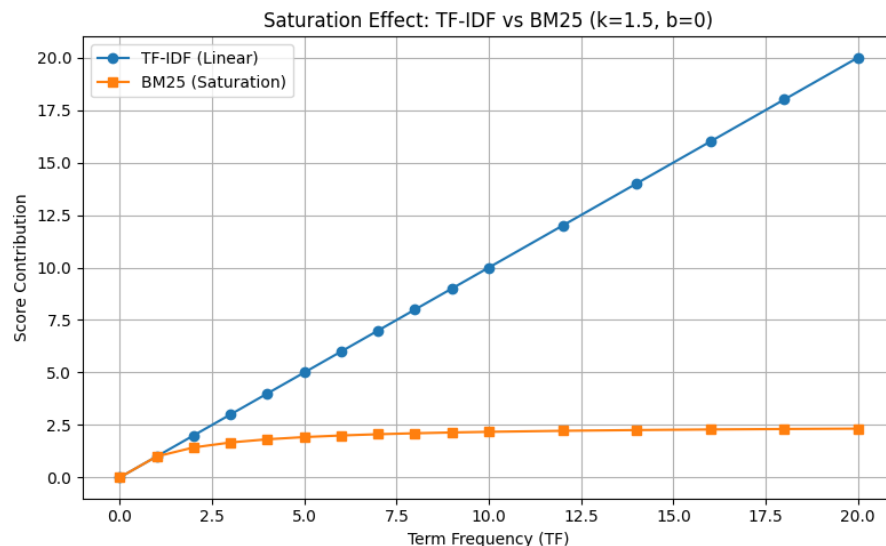


Advanced Chunking

Best Matching (BM25)

- BM25 is an extension of TF-IDF that introduces additional parameters to make the weighting flexible

$$\text{score}(D, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)}$$



- Term frequency helps but only up to a point
 - Early occurrences of a term indicate topic importance
 - Later occurrences often add limited new information
- BM25 models this with a **saturation TF function**
- Leads to more robust ranking than TF-IDF

Advanced Chunking

Best Matching (BM25)

- How to properly adjust hyperparams?

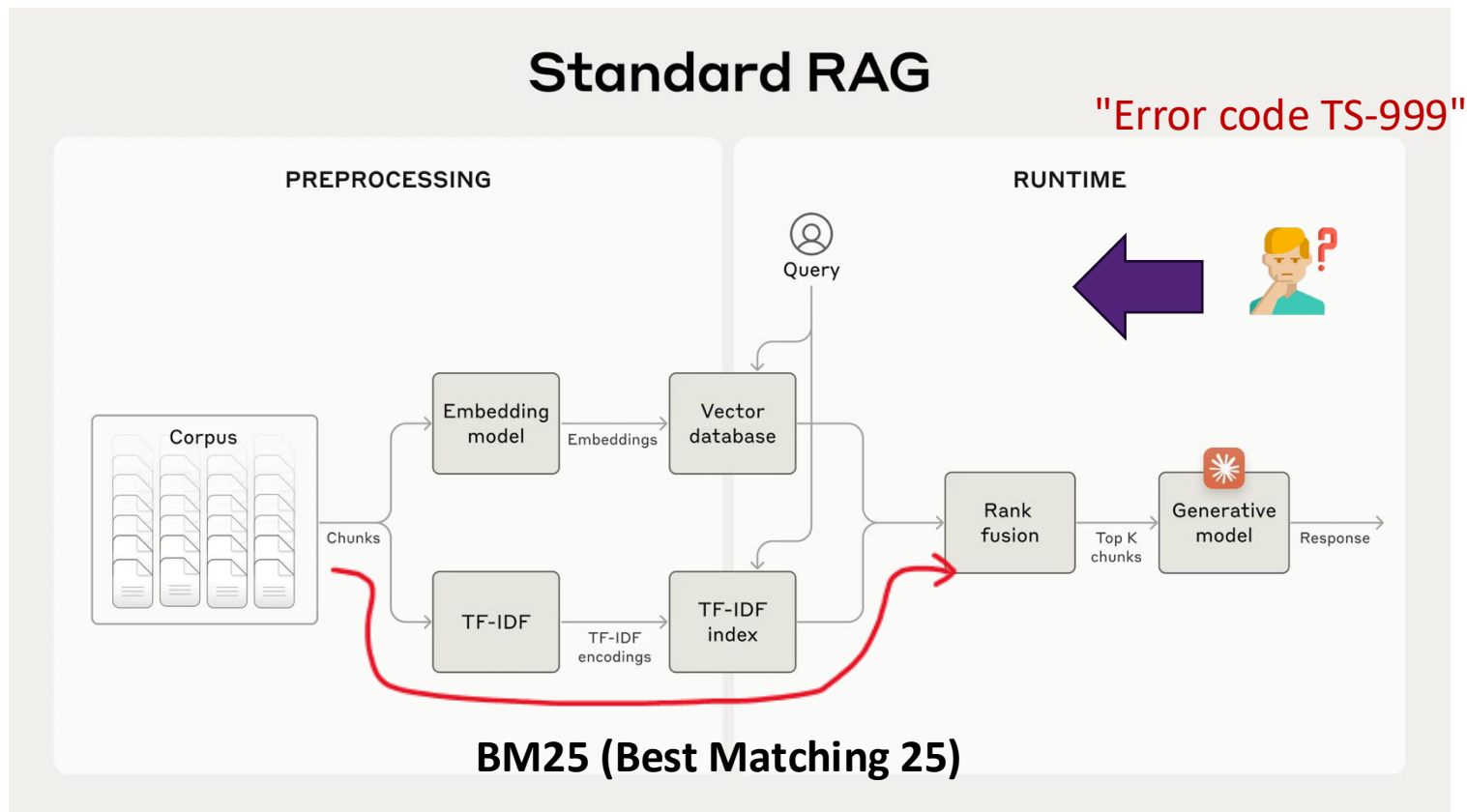
$$\text{score}(D, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)}$$

- k_1 controls term frequency saturation [1.2, 2.0]
 - Smaller k_1 → documents are noisy or repetitive
 - Larger k_1 → documents are short and term frequency is a strong relevance signal
- b controls doc length normalization [0.3, 0.9]
 - Smaller b → stronger penalty/adjustment for long documents
 - Larger b → weaker length normalization

Advanced Chunking

- All chunking methods still inevitably lose critical information...

Contextual Retrieval (Anthropic, 2024)

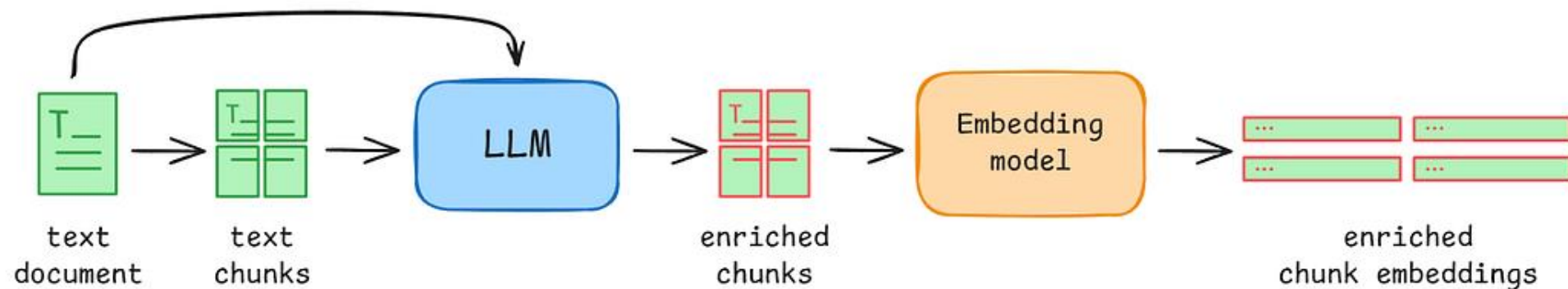


1. Break down the knowledge base (the "corpus" of documents) into smaller **chunks** of text, usually no more than a few hundred tokens;
2. Create **TF-IDF** encodings and semantic embeddings for these chunks;
3. Use **BM25** to find top chunks based on exact matches;
4. Use **embeddings** to find top chunks based on semantic similarity;
5. **Combine and deduplicate results from (3) and (4) using rank fusion techniques;**
6. Add the top-K chunks to the **prompt** to generate the response.

Advanced Chunking

- All chunking methods still inevitably lose critical information...

Contextual Retrieval (Anthropic, 2024)



[Orig Chunk]" The company's revenue grew by 3% over the previous quarter."



[LLM] The company is growing fast...

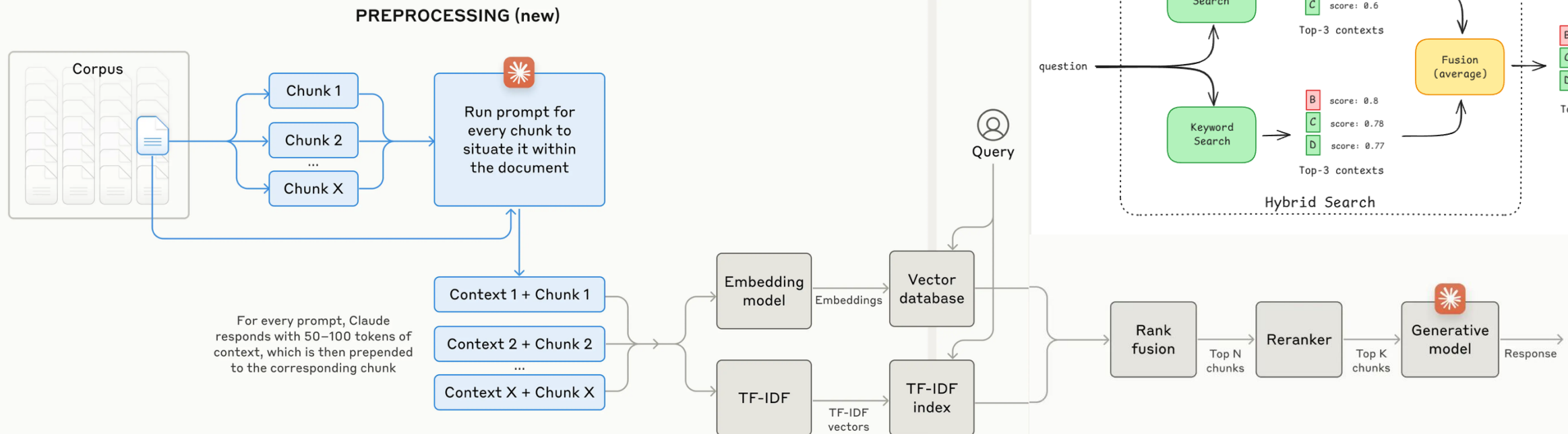
prepend <context> to each chunk?

```
<document>
{{WHOLE_DOCUMENT}}
</document>
Here is the chunk we want to situate within the whole document
<chunk>
{{CHUNK_CONTENT}}
</chunk>
Please give a short succinct context to situate this chunk within
the overall document for the purposes of improving search retrieval
of the chunk. Answer only with the succinct context and nothing
else.
```

Advanced Chunking

- In hybrid search, results from both methods are merged and weighted to balance their contributions.
- Contextual Retrieval (Anthropic, 2024)**

Contextual Retrieval Preprocessing



Advanced RAG

1) Pre-retrieval Process

Q: How much is the fee for a H-1B in 2026?

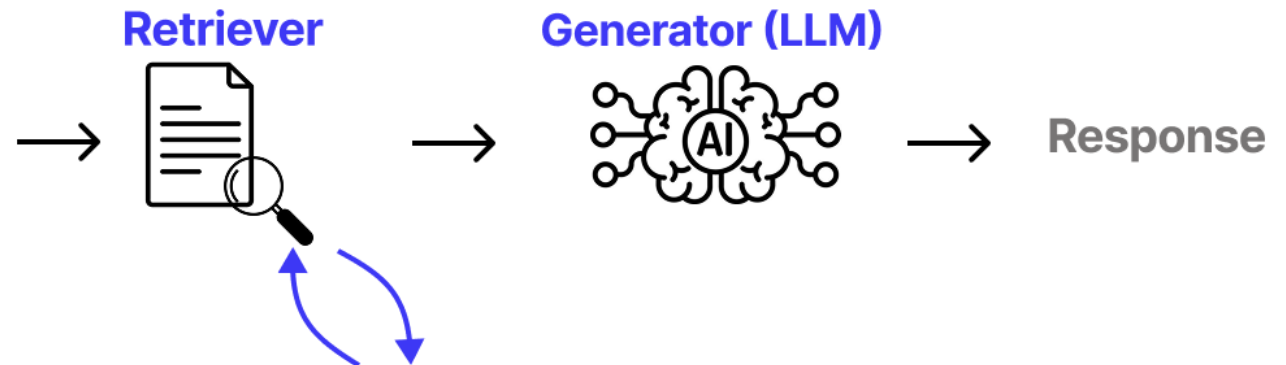


- Query expansion
- Query decomposition
- Query rewriting

Q1. What are the official H-1B petition filing fees before 2026?

Q2. Is there an additional one-time payment required for new H-1B petitions in 2026?

Q3. What are the total H-1B petition fees in 2026?



[Knowledge base]

Relevant document 

By 2026, a new one-time \$100,000 payment is imposed on all new H-1B petitions, in addition to the existing fees ...

Relevant document 

In 2024, the H-1B visa process requires a \$205 consular application fee along with multiple USCIS petition fees ...

Irrelevant document

By 2026, U.S. technology, biotech, and semiconductor sectors are projected to depend heavily on H-1B professionals...

Advanced RAG

1) Pre-retrieval Process

Q: How much is the fee for a H-1B in 2026?



- Query expansion
- Query decomposition
- Query rewriting

Q1. What are the official H-1B petition filing fees before 2026?

Q2. Is there an additional one-time payment required for new H-1B petitions in 2026?

Q3. What are the total H-1B petition fees in 2026?

[Knowledge base]

Relevant document 1

By 2026, a new one-time \$100,000 payment is imposed on all new H-1B petitions, in addition to the existing fees ...

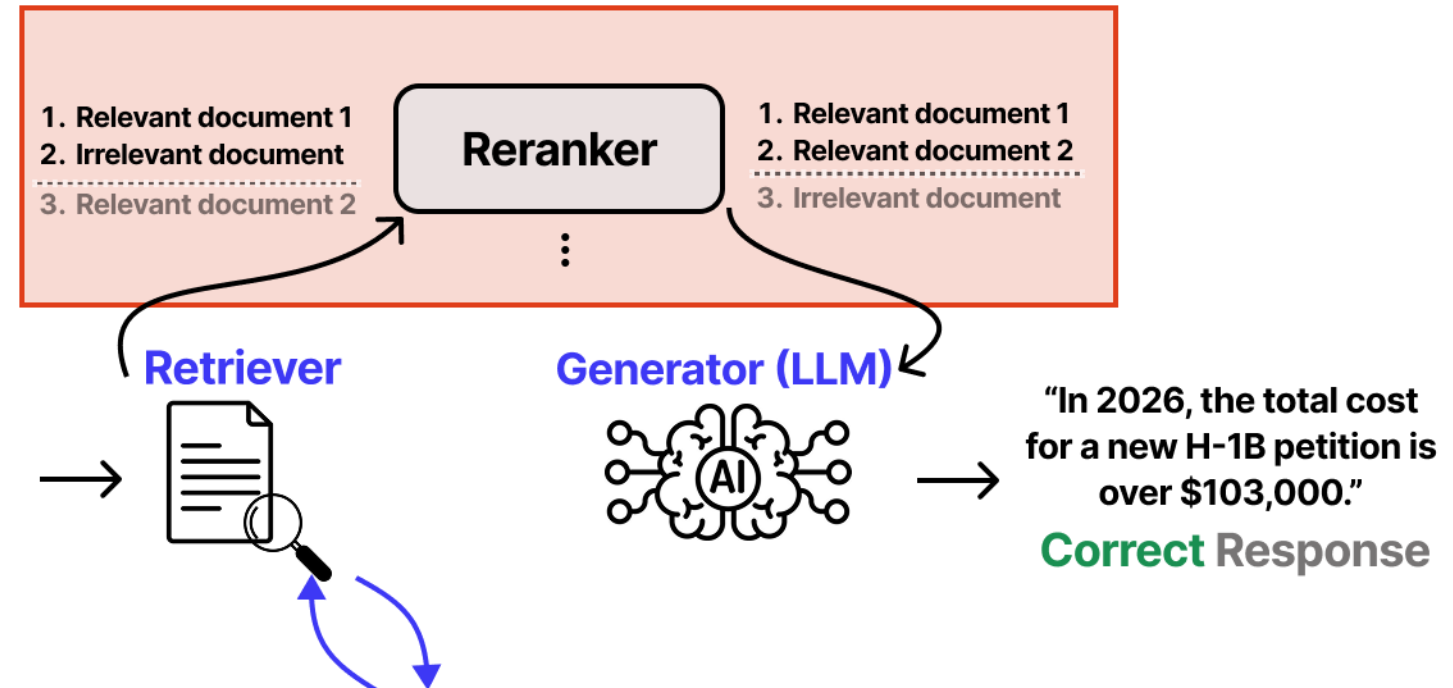
Relevant document 2

In 2024, the H-1B visa process requires a \$205 consular application fee along with multiple USCIS petition fees ...

Irrelevant document

By 2026, U.S. technology, biotech, and semiconductor sectors are projected to depend heavily on H-1B professionals...

2) Post-retrieval Process

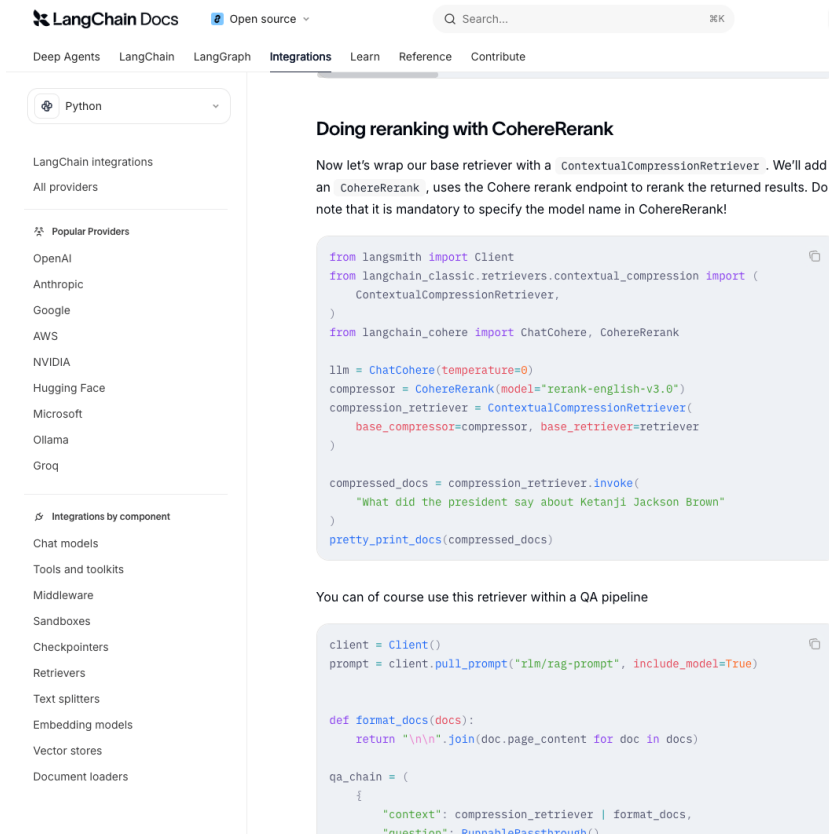


Reranker in RAG

- Regardless of the embedding model used, re-ranking demonstrates a higher hit rate and MRR

Embedding	WithoutReranker		bge-reranker-base		bge-reranker-large		Cohere-Reranker	
	Hit Rate	MRR	Hit Rate	MRR	Hit Rate	MRR	Hit Rate	MRR
OpenAI	0.876404	0.718165	0.91573	0.832584	0.910112	0.855805	0.926966	0.86573
bge-large	0.752809	0.597191	0.859551	0.805243	0.865169	0.816011	0.876404	0.822753
llm-embedder	0.814607	0.587266	0.870787	0.80309	0.876404	0.824625	0.882022	0.830243
Cohere-v2	0.780899	0.570506	0.876404	0.798127	0.876404	0.825281	0.876404	0.815543
Cohere-v3	0.825843	0.624532	0.882022	0.806086	0.882022	0.834644	0.88764	0.836049
Voyage	0.831461	0.68736	0.926966	0.837172	0.91573	0.858614	0.91573	0.851217
JinaAI-Small	0.831461	0.614045	0.91573	0.843071	0.926966	0.857303	0.926966	0.868633
JinaAI-Base	0.848315	0.68221	0.938202	0.846348	0.938202	0.868539	0.932584	0.873689
Google-PaLM	0.865169	0.719476	0.910112	0.833708	0.910112	0.85309	0.910112	0.855712

- Can GPT works for reranking?



LangChain Docs Open source

Deep Agents LangChain LangGraph Integrations Learn Reference Contribute

Python

LangChain Integrations

All providers

Popular Providers

OpenAI

Anthropic

Google

AWS

NVIDIA

Hugging Face

Microsoft

Ollama

Groq

Integrations by component

Chat models

Tools and toolkits

Middleware

Sandboxes

Checkpointers

Retrievers

Text splitters

Embedding models

Vector stores

Document loaders

Doing reranking with CohereRerank

Now let's wrap our base retriever with a `ContextualCompressionRetriever`. We'll add an `CohereRerank`, uses the Cohere rerank endpoint to rerank the returned results. Do note that it is mandatory to specify the model name in `CohereRerank`!

```
from langsmith import Client
from langchain_classic.retrievers.contextual_compression import (
    ContextualCompressionRetriever,
)
from langchain_cohere import ChatCohere, CohereRerank

llm = ChatCohere(temperature=0)
compressor = CohereRerank(model="rerank-english-v3.0")
compression_retriever = ContextualCompressionRetriever(
    base_compressor=compressor, base_retriever=retriever
)

compressed_docs = compression_retriever.invoke(
    "What did the president say about Ketanji Jackson Brown"
)
pretty_print_docs(compressed_docs)
```

You can of course use this retriever within a QA pipeline

```
client = Client()
prompt = client.pull_prompt("rlm/rag-prompt", include_model=True)

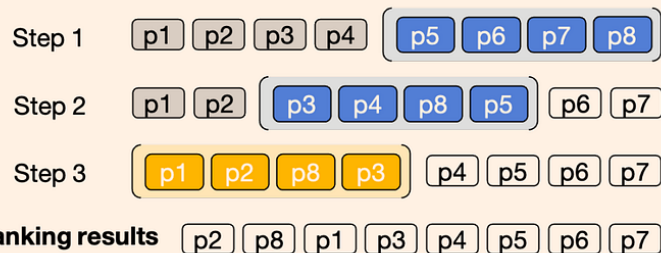
def format_docs(docs):
    return "\n\n".join(doc.page_content for doc in docs)

qa_chain = (
    {
        "context": compression_retriever | format_docs,
        "question": RunnablePassthrough(),
    }
```

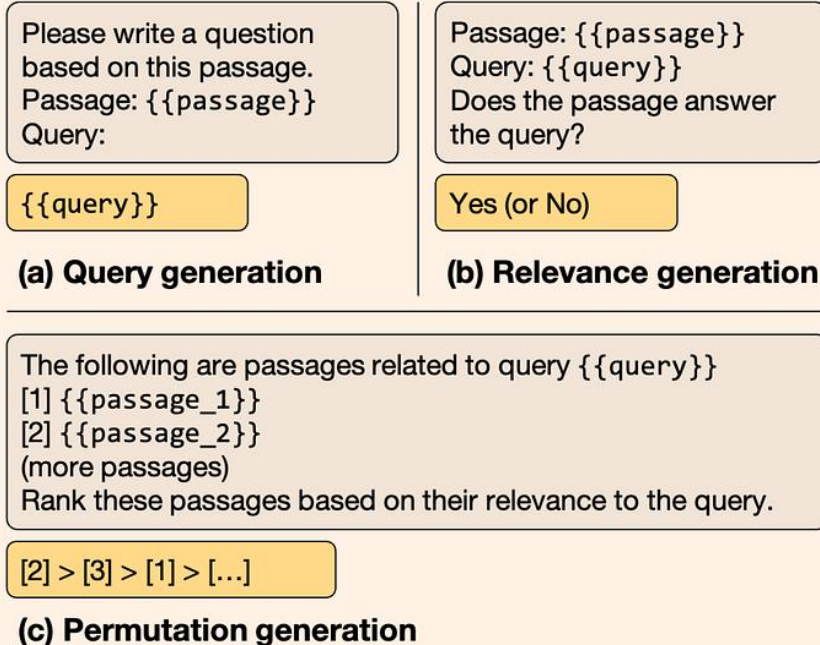

Reranker in RAG

- The idea of **RankGPT** is to perform zero-shot listwise passage re-ranking using LLM (such as ChatGPT or GPT-4 or other LLMs). It applies a **permutation generation** approach and **sliding window** strategy to efficiently re-rank passages.

Typically the number of candidate documents is very large while the input to LLM is limited...



Sliding window



Is ChatGPT Good at Search? Investigating Large Language Models as Re-Ranking Agents

Weiwei Sun^{1*} Lingyong Yan² Xinyu Ma² Shuaiqiang Wang²
Pengjie Ren¹ Zhumin Chen¹ Dawei Yin^{2†} Zhaochun Ren^{3†}

¹Shandong University, Qingdao, China ²Baidu Inc., Beijing, China

³Leiden University, Leiden, The Netherlands

{sunweiwei, lingyong, xinyuma2016, shqiang.wang}@gmail.com

{renpengjie, chenzhumin}@sdu.edu.cn yindawei@acm.org

z.ren@liacs.leidenuniv.nl

Abstract

Large Language Models (LLMs) have demonstrated remarkable zero-shot generalization across various language-related tasks, including search engines. However, existing work utilizes the generative ability of LLMs for Information Retrieval (IR) rather than direct passage ranking. The discrepancy between the pre-training objectives of LLMs and the ranking objective poses another challenge. In this paper, we first investigate generative LLMs such as ChatGPT and GPT-4 for relevance ranking in IR. Surprisingly, our experiments reveal that properly instructed LLMs can deliver competitive, even superior results to state-of-the-art supervised methods on popular IR benchmarks. Furthermore, to address concerns about data contamination of LLMs, we collect a new test set called NovelEval, based on the latest knowledge and aiming to verify the model's ability to rank unknown knowledge. Finally, to improve efficiency in real-world applications, we delve into the potential for distilling the ranking capabilities of ChatGPT into small specialized models using a permutation distillation scheme. Our evaluation results turn out that

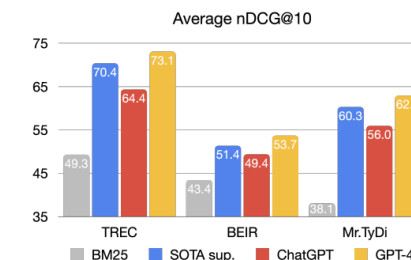


Figure 1: Average results of ChatGPT and GPT-4 (zero-shot) on passage re-ranking benchmarks (TREC, BEIR, and Mr.TyDi), compared with BM25 and previous best-supervised systems (SOTA sup., e.g., monoT5 (Nogueira et al., 2020)).

As one of the most successful AI applications, Information Retrieval (IR) systems satisfy user requirements through several pipelined sub-modules, such as passage retrieval and re-ranking (Lin et al., 2020). Most previous methods heavily rely on manual supervision signals, which require significant human effort and demonstrate weak generaliz-

Reranker in RAG

- The idea of **RankGPT** is to perform zero-shot listwise passage re-ranking using LLM (such as ChatGPT or GPT-4 or other LLMs). It applies a **permutation generation** approach and **sliding window** strategy to efficiently re-rank passages.

optimal performance within specific inference constraints This is achieved by tr
more tokens...

Start reranking...

Node ID: 47ba3955-c6f8-4f28-a3db-f3222b3a09cd

Score: 0.8621633467539512

Text: TinyLlama: An Open-Source Small Language Model

Peiyuan Zhang*Guangtao Zeng*Tianduo Wang Wei Lu

StatNLP Research Group

Singapore University of Technology and Design

{peiyuan_zhang, tianduo_wang, @sutd.edu.sg">luwei}@sutd.edu.sg

guangtao_zeng@mymail.sutd.edu.sg

Abstract

We present TinyLlama, a compact 1.1B language model pretrained on around 1
trillion tokens for approximately 3 epochs. Building on the architecture and tok
enizer of Llama 2 (Touvron et al., 2023b), TinyLlama leverages various advances
contr...

Node ID: 17cd9896-473c-47e0-8419-16b4ac615a59

Score: 0.8343984516104476

Text: Although these works show a clear preference on large models, the potentia
models with larger dataset remains under-explored. Instead of training compute-o
models, Touvron et al. (2023a) highlight the importance of the inference budget,
solely on training compute-optimal language models. Inference-optimal language m
optimal performance within specific inference constraints This is achieved by tr
more tokens...

Node ID: 20de8234-a668-442d-8495-d39b156b44bb

Score: 0.8703492815379594

Text: 4 Conclusion

In this paper, we introduce TinyLlama, an open-source, small-scale language mode
transparency in the open-source LLM pre-training community, we have released all
mation, including our pre-training code, all intermediate model checkpoints, and
data processing steps. With its compact architecture and promising performance,
enable end-user applications on mobile devices, and serve as a lightweight platf
w...

```
from llama_index.postprocessor import RankGPTRerank
from llama_index.llms import OpenAI
reranker = RankGPTRerank(
    top_n = 3,
    llm = OpenAI(model="gpt-3.5-turbo-16k"),
    # verbose=True,
)
```



LlamaIndex

LlamaIndex

Longllmlingua

Metadata replacement

Mixedbreadai rerank

Nvidia rerank

Openvino rerank

Pinecone native rerank

Presidio

Prev next

Rankgpt rerank

Rankllm rerank

Sbert rerank

Sentence optimizer

Siliconflow rerank

Similarity

Tei rerank

Time weighted

Voyageai rerank

Rankgpt rerank

RankGPTRerank

Bases: [BaseNodePostprocessor](#)

RankGPT-based reranker.

<> Source code in [llama-index-integrations/postprocessor/llama-index-rerank/llama_index/postprocessor/rankgpt_rerank/base.py](#)

options: members: - RankGPTRerank

Reranker in RAG

- Any reranker for free? -> **Reciprocal Rank Fusion (RRF)**
 - Combines ranked lists from multiple retrievers or ranking methods (easy, commonly set k to 60)
 - Uses **rank positions** instead of **raw scores**

$$\text{RRF}(d) = \sum_{i=1}^N \frac{1}{k + \text{rank}_i(d)}$$

Dense Search (List A)

Doc	Rank
D1	1
D2	2
D3	3

Sparse Search (List B)

Doc	Rank
D3	1
D1	5
D4	2

For Document D1

Dense rank = 1

Sparse rank = 5

$$\text{Score} = \frac{1}{60 + 1} + \frac{1}{60 + 5} = 0.01639 + 0.01538 = 0.03177$$

For Document D3

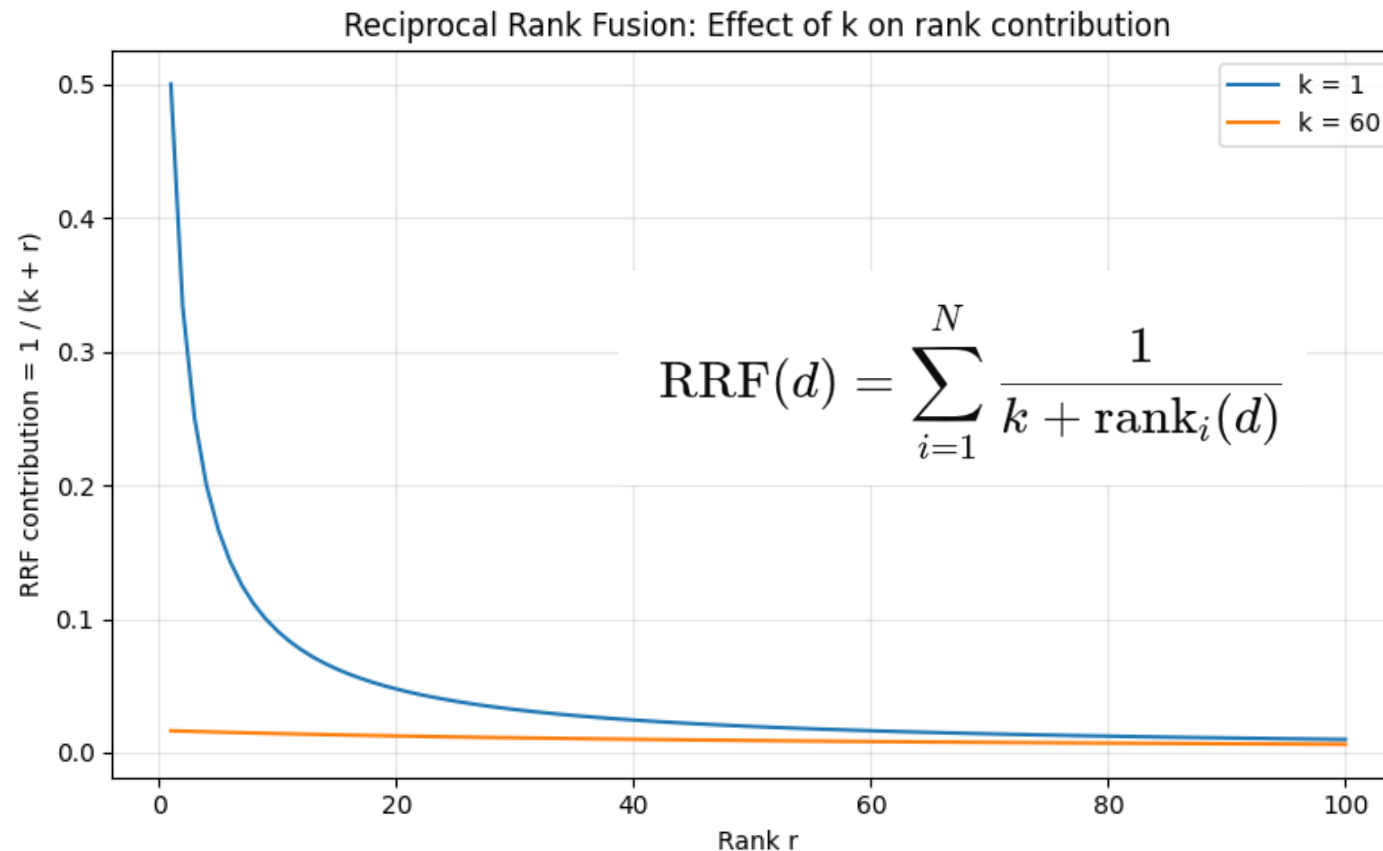
Dense rank = 3

Sparse rank = 1

$$= \frac{1}{63} + \frac{1}{61} = 0.01587 + 0.01639 = 0.03226$$

Reranker in RAG

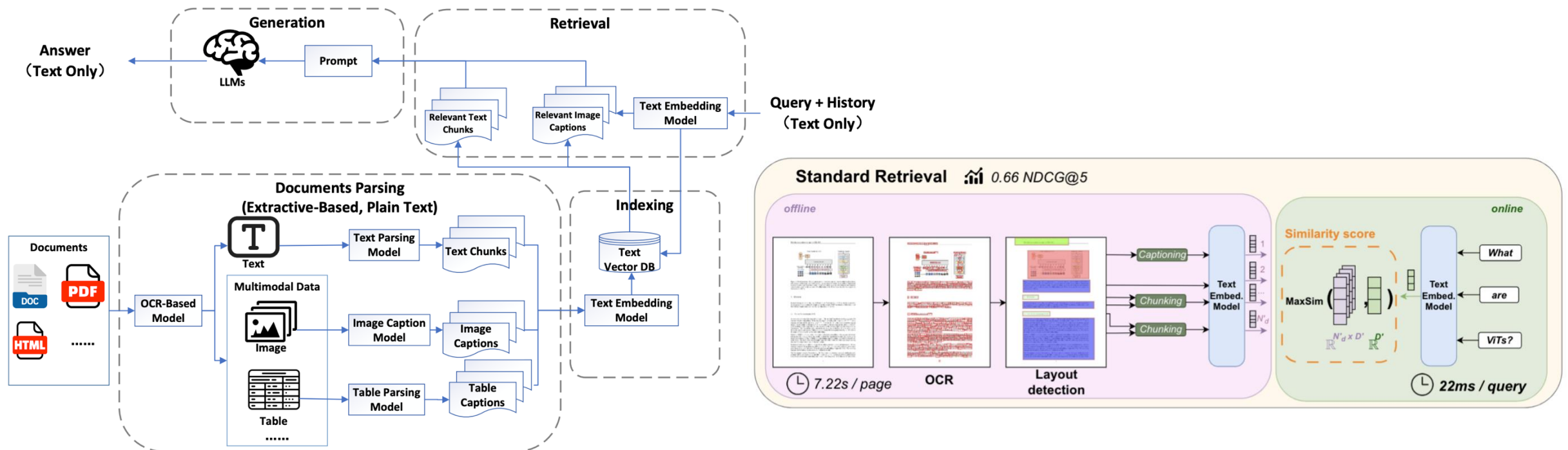
- Any reranker for free? -> **Reciprocal Rank Fusion (RRF)**
 - Combines ranked lists from multiple retrievers or ranking methods (easy, commonly set k to 60)
 - Uses **rank positions** instead of **raw scores**



Multimodal RAG Roadmap

MRAG 1.0 (pseudo-MRAG):

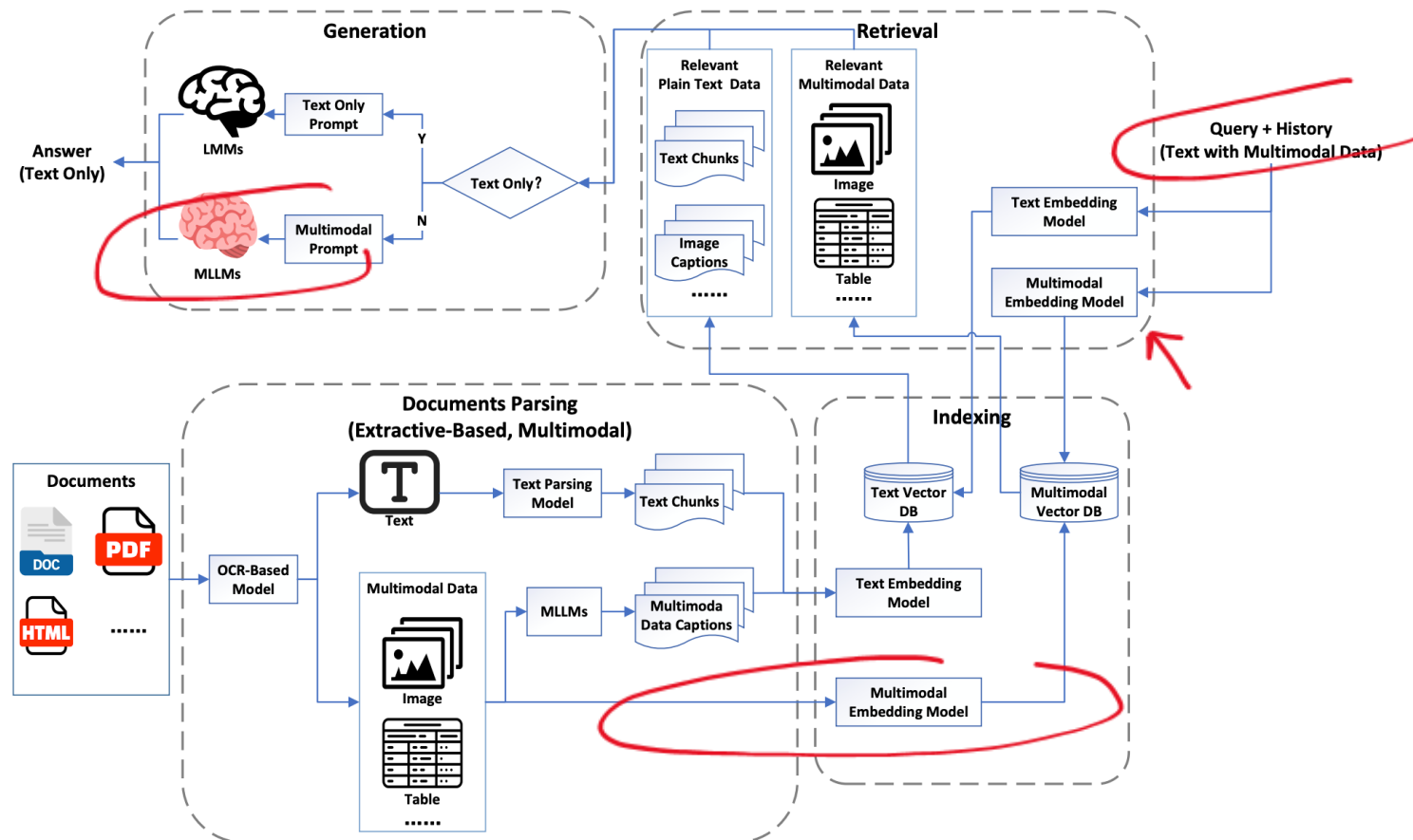
- convert images/tables/other media into **captions or OCR text**,
- -> text-style RAG pipeline
- Easy to build, but modality conversion causes information loss and retrieval bottlenecks.



Multimodal RAG Roadmap

MRAG 2.0 (true multimodal):

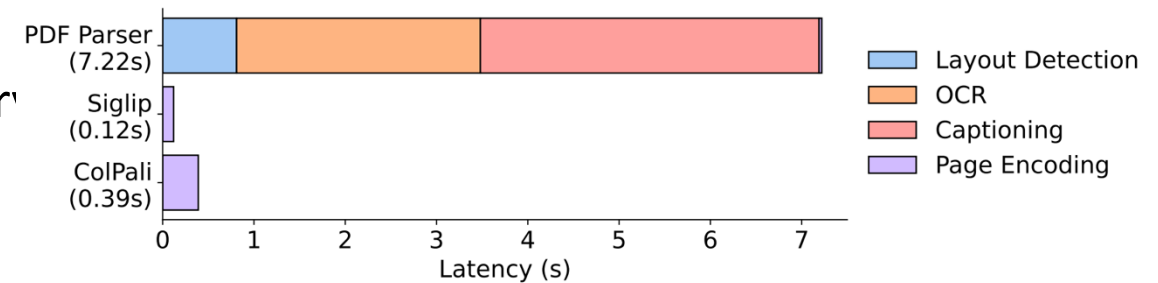
- preserve the original multimodal data in the knowledge base, support **multimodal retrieval**,
- and use **MLLMs** to generate answers directly from multimodal evidence.



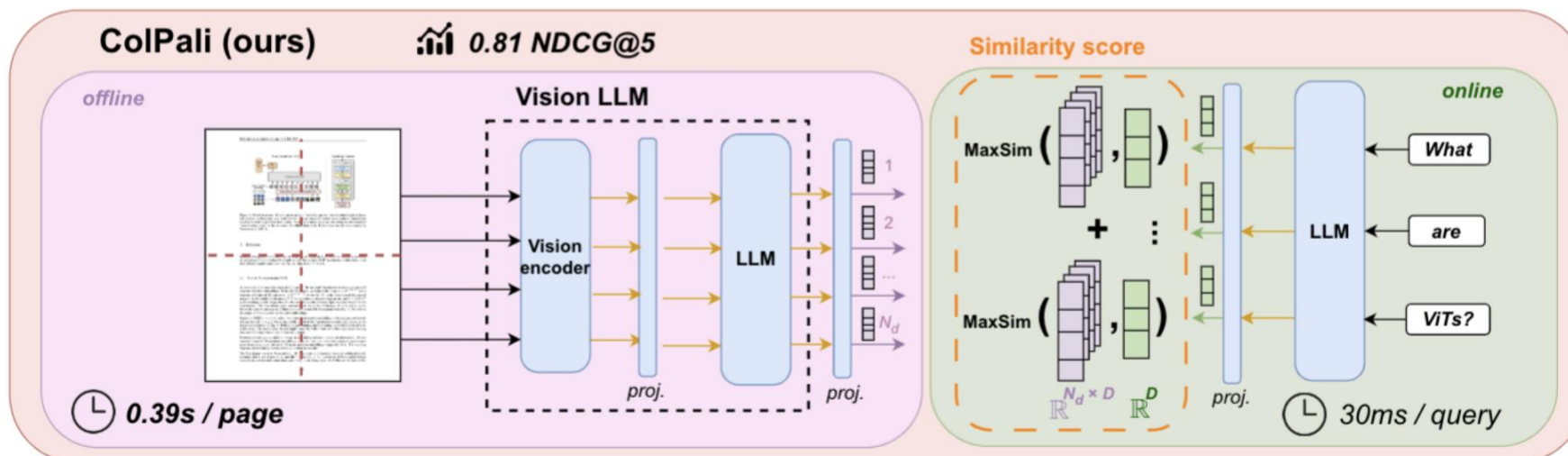
Multimodal RAG Roadmap

ColPali

- IR benchmarks measures clean pre-processed texts
- In practical industrial settings we don't have the luxury of pre-processed datasets.
- Performance bottleneck usually not in embedding model performance or chunking, but in the **data ingestion** pipeline



PaliGemma (VLM): finetuned to produce embeddings (remove head)



Contrastive Loss

(similar to CLIP)

$$\mathcal{L} = -\frac{1}{b} \sum_{k=1}^b \log \left[\frac{\exp(s_k^+)}{\exp(s_k^+) + \exp(s_k^-)} \right]$$

Multimodal RAG Roadmap

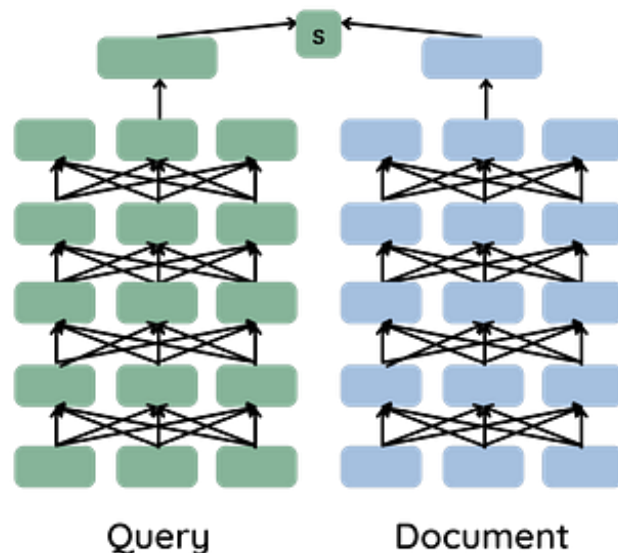
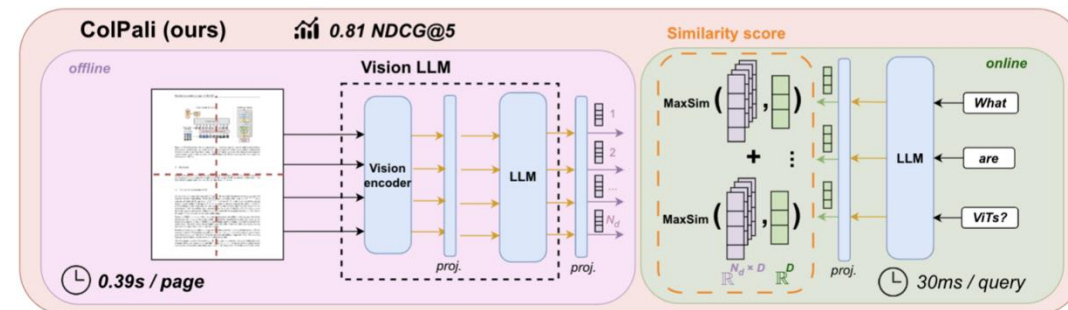
ColPali

- ✓ **Match without text extraction**
Do not have to map complex formats to the text domain
- ✓ **Engineering Simplicity**
Reduced document processing pipeline complexity
- ✓ **Avoid OCR**
Use the VLM OCR capabilities
- ✓ **Better relevance**
Outcompete traditional extract methods
- ✓ **No Layout Detection**
Layout is encoded by the VLM
- ✓ **A future direction for document retrieval**
Can you print the document, then you can index it with the ColPali approach

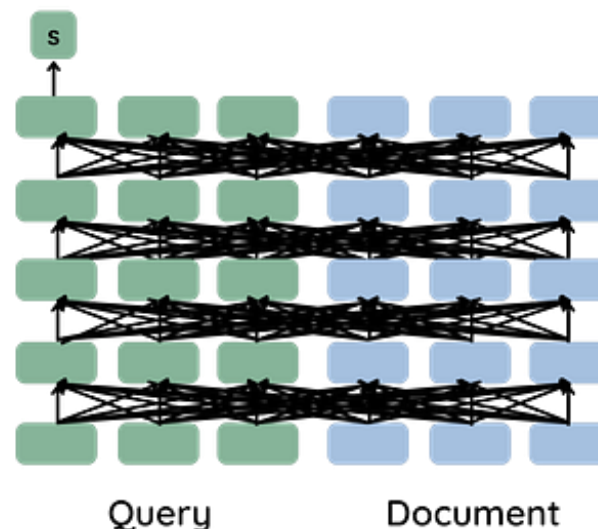
Multimodal RAG Roadmap

Late Interaction in ColPali

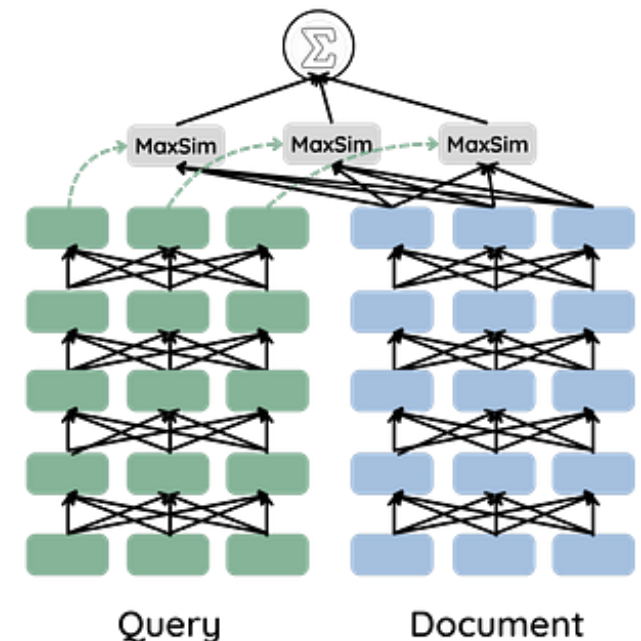
- Middle step between embedding and reranking



Representation-based similarity



All-in-all interaction (Reranker)



$$\mathbf{E}_q \in \mathbb{R}^{N_q \times D}$$

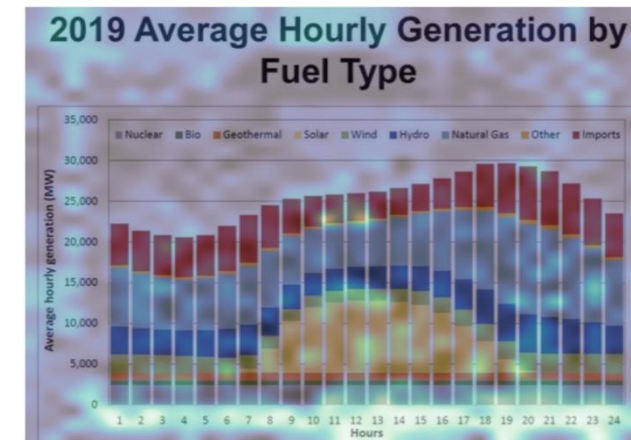
$$\mathbf{E}_d \in \mathbb{R}^{N_d \times D}$$

Late Interaction (ColBERT)

$$LI(q, d) = \sum_{i \in [1, N_q]} \max_{j \in [1, N_d]} \langle \mathbf{E}_q^{(i)} | \mathbf{E}_d^{(j)} \rangle$$

Multimodal RAG Roadmap

	ArxivQ	DocQ	InfoQ	TabF	TATQ	Shift	AI	Energy	Gov.	Health.	Avg.
Unstructured text-only											
- BM25	-	34.1	-	-	44.0	59.6	90.4	78.3	78.8	82.6	-
- BGE-M3	-	28.4 _{↓5.7}	-	-	36.1 _{↓7.9}	68.5 _{↑8.9}	88.4 _{↓2.0}	76.8 _{↓1.5}	77.7 _{↓1.1}	84.6 _{↑2.0}	-
Unstructured + OCR											
- BM25	31.6	36.8	62.9	46.5	62.7	64.3	92.8	85.9	83.9	87.2	65.5
- BGE-M3	31.4 _{↓0.2}	25.7 _{↓11.1}	60.1 _{↓2.8}	70.8 _{↑24.3}	50.5 _{↓12.2}	73.2 _{↑8.9}	90.2 _{↓2.6}	83.6 _{↓2.3}	84.9 _{↑1.0}	91.1 _{↑3.9}	66.1 _{↑0.6}
Unstructured + Captioning											
- BM25	40.1	38.4	70.0	35.4	61.5	60.9	88.0	84.7	82.7	89.2	65.1
- BGE-M3	35.7 _{↓4.4}	32.9 _{↓5.4}	71.9 _{↑1.9}	69.1 _{↑33.7}	43.8 _{↓17.7}	73.1 _{↑12.2}	88.8 _{↑0.8}	83.3 _{↓1.4}	80.4 _{↓2.3}	91.3 _{↑2.1}	67.0 _{↑1.9}
Contrastive VLMs											
Jina-CLIP	25.4	11.9	35.5	20.2	3.3	3.8	15.2	19.7	21.4	20.8	17.7
Nomic-vision	17.1	10.7	30.1	16.3	2.7	1.1	12.9	10.9	11.4	15.7	12.9
SigLIP (Vanilla)	43.2	30.3	64.1	58.1	26.2	18.7	62.5	65.7	66.1	79.1	51.4
Ours											
SigLIP (Vanilla)	43.2	30.3	64.1	58.1	26.2	18.7	62.5	65.7	66.1	79.1	51.4
BiSigLIP (+fine-tuning)	58.5 _{↑15.3}	32.9 _{↑2.6}	70.5 _{↑6.4}	62.7 _{↑4.6}	30.5 _{↑4.3}	26.5 _{↑7.8}	74.3 _{↑11.8}	73.7 _{↑8.0}	74.2 _{↑8.1}	82.3 _{↑3.2}	58.6 _{↑7.2}
BiPali (+LLM)	56.5 _{↓-2.0}	30.0 _{↓-2.9}	67.4 _{↓-3.1}	76.9 _{↑14.2}	33.4 _{↑2.9}	43.7 _{↑17.2}	71.2 _{↓-3.1}	61.9 _{↓-11.7}	73.8 _{↓-0.4}	73.6 _{↓-8.8}	58.8 _{↑0.2}
ColPali (+Late Inter.)	79.1 _{↑22.6}	54.4 _{↑24.5}	81.8 _{↑14.4}	83.9 _{↑7.0}	65.8 _{↑32.4}	73.2 _{↑29.5}	96.2 _{↑25.0}	91.0 _{↑29.1}	92.7 _{↑18.9}	94.4 _{↑20.8}	81.3 _{↑22.5}



Query: "Which hour of the day had the highest overall electricity generation in 2019?"



ViDoRe: Visual Document Retrieval

ViDoRe V3: our new benchmark release!

Developed by ILLUIN Technology and with contributions from NVIDIA, it is the most diverse visual document retrieval benchmark to date for enterprise applications. It includes 10 datasets, 26,000+ pages, 3000+ queries in 6 languages. Check out: [the paper](#) or [the blogpost](#)!

This organization contains models, datasets, benchmarks and code released with the ViDoRe project by Illuin Technology.

• **Leaderboard:**

• [ViDoRe Leaderboard](#)

• **Benchmarks:**

• ViDoRe V1 ([paper](#), [blogpost](#), [dataset collection](#))

• ViDoRe V2 ([paper](#), [blogpost](#), [dataset collection](#))

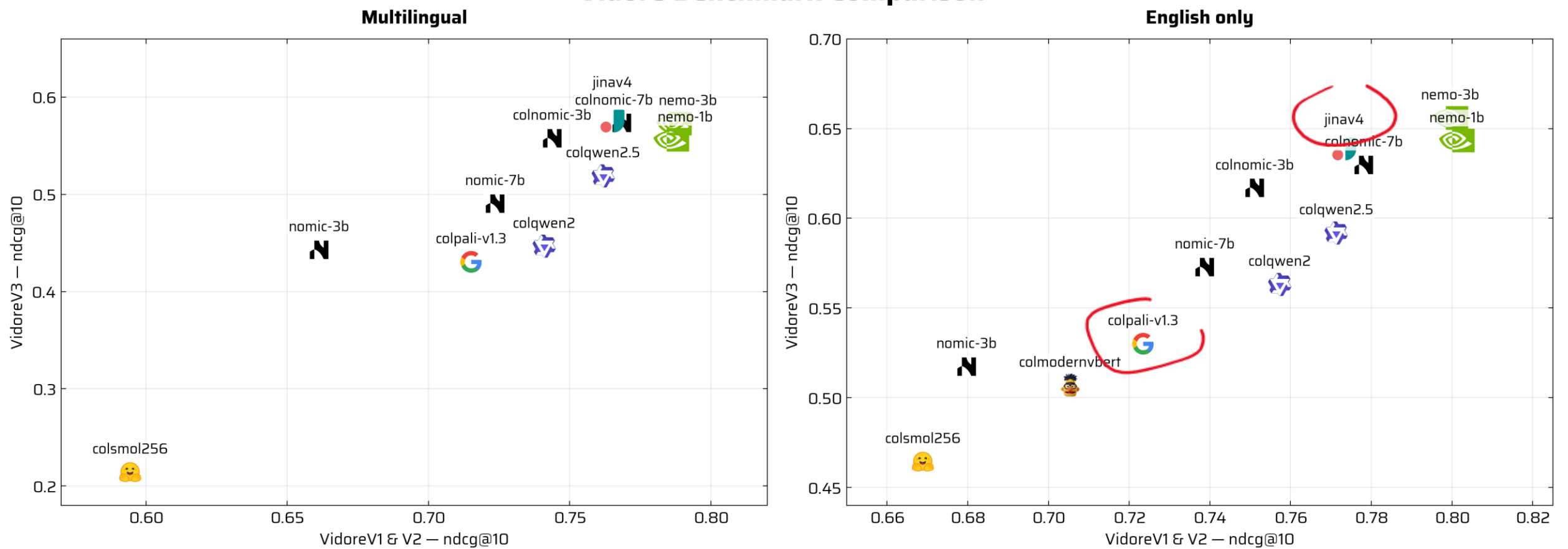
• ViDoRe V3 ([paper](#), [blogpost](#), [dataset collection](#))

• **Models:**

<https://huggingface.co/vidore>

Multimodal RAG Roadmap

Vidore Benchmark Comparison



jinav4: jina-embeddings-v4

colnomic-3b: colnomic-embed-3b

colnomic-7b: colnomic-embed-7b

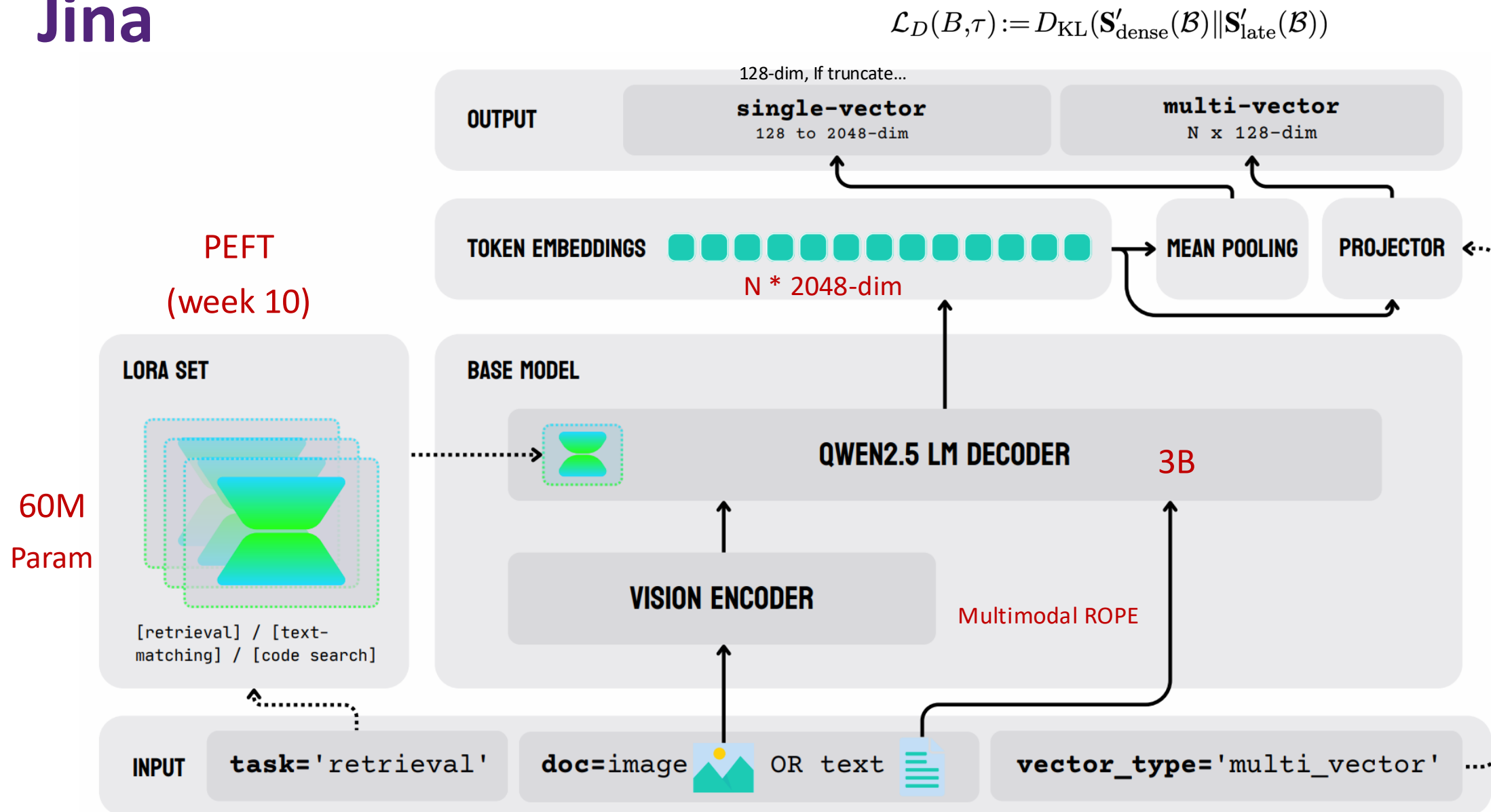
nomic-3b: nomic-embed-3b

nomic-7b: nomic-embed-7b

nemo-3b: nemo-colembed-3b

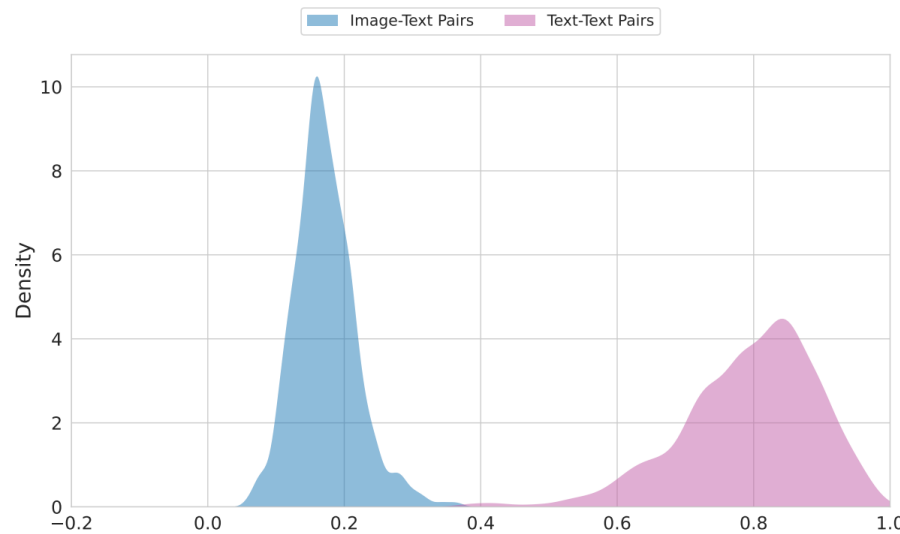
nemo-1b: nemo-colembed-1b

Jina



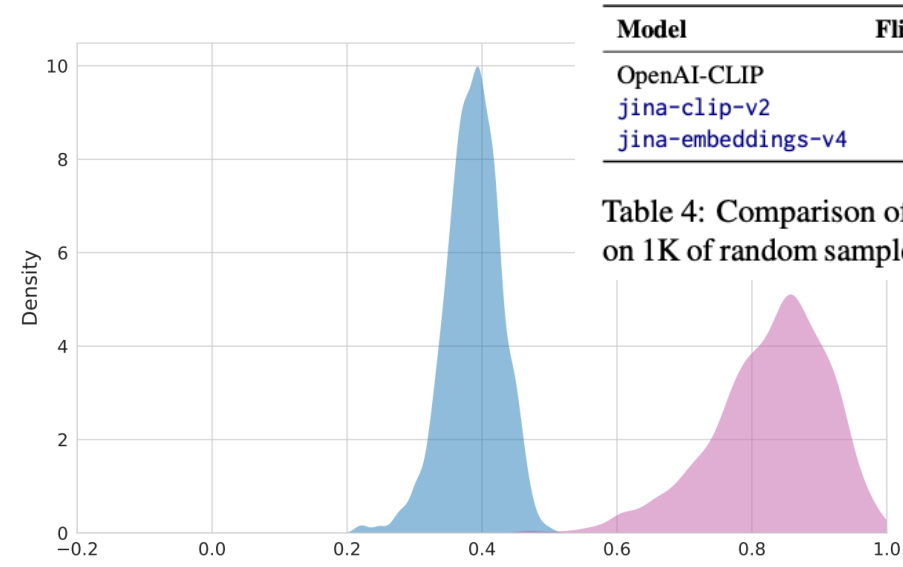
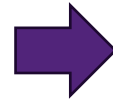
Jina

		Text Retrieval						Image Retrieval					
		Flickr30k			MSCOCO			Flickr30k			MSCOCO		
		R@1	R@5	R@10	R@1	R@5	R@10	R@1	R@5	R@10	R@1	R@5	R@10
Finetune	Unicoder-VL ^a	86.2	96.3	99.0	62.3	87.1	92.8	71.5	90.9	94.9	46.7	76.0	85.3
	Uniter ^b	87.3	98.0	99.2	65.7	88.6	93.8	75.6	94.1	96.8	52.9	79.9	88.0
	VILLA ^c	87.9	97.5	98.8	-	-	-	76.3	94.2	96.8	-	-	-
	Oscar ^d	-	-	-	73.5	92.2	96.0	-	-	-	57.5	82.8	89.8
	ERNIE-ViL ^e	88.7	98.0	99.2	-	-	-	76.7	93.6	96.4	-	-	-
Zero-Shot	Visual N-Grams ^f	15.4	35.7	45.1	8.7	23.1	33.3	8.8	21.2	29.9	5.0	14.5	21.9
	ImageBERT ^g	-	-	-	44.0	71.2	80.4	-	-	-	32.3	59.0	70.2
	Unicoder-VL ^a	64.3	86.8	92.3	-	-	-	48.4	76.0	85.2	-	-	-
	Uniter ^b	83.6	95.7	97.7	-	-	-	68.7	89.2	93.9	-	-	-
	CLIP	88.0	98.7	99.4	58.4	81.5	88.1	68.7	90.6	95.2	37.8	62.4	72.2

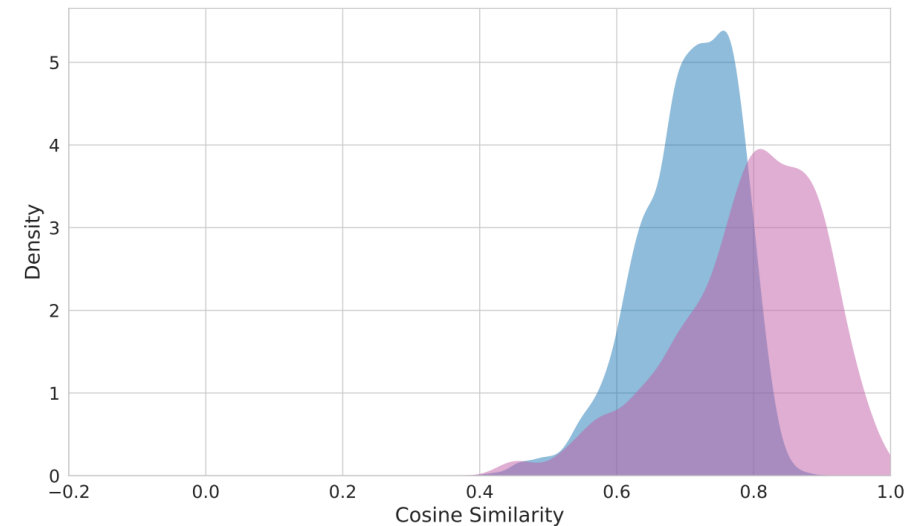


OpenAI CLIP

Modality Gap: examining the distribution of pairwise cosine similarities of matching **image-text** pairs versus matching **text-text** pairs



Jina-CLIP-v2



Jina-embedding-v4

Model	Flickr30K	MSCOCO	CIFAR-100
OpenAI-CLIP	0.15	0.14	0.2
jina-clip-v2	0.38	0.37	0.32
jina-embeddings-v4	0.71	0.72	0.56

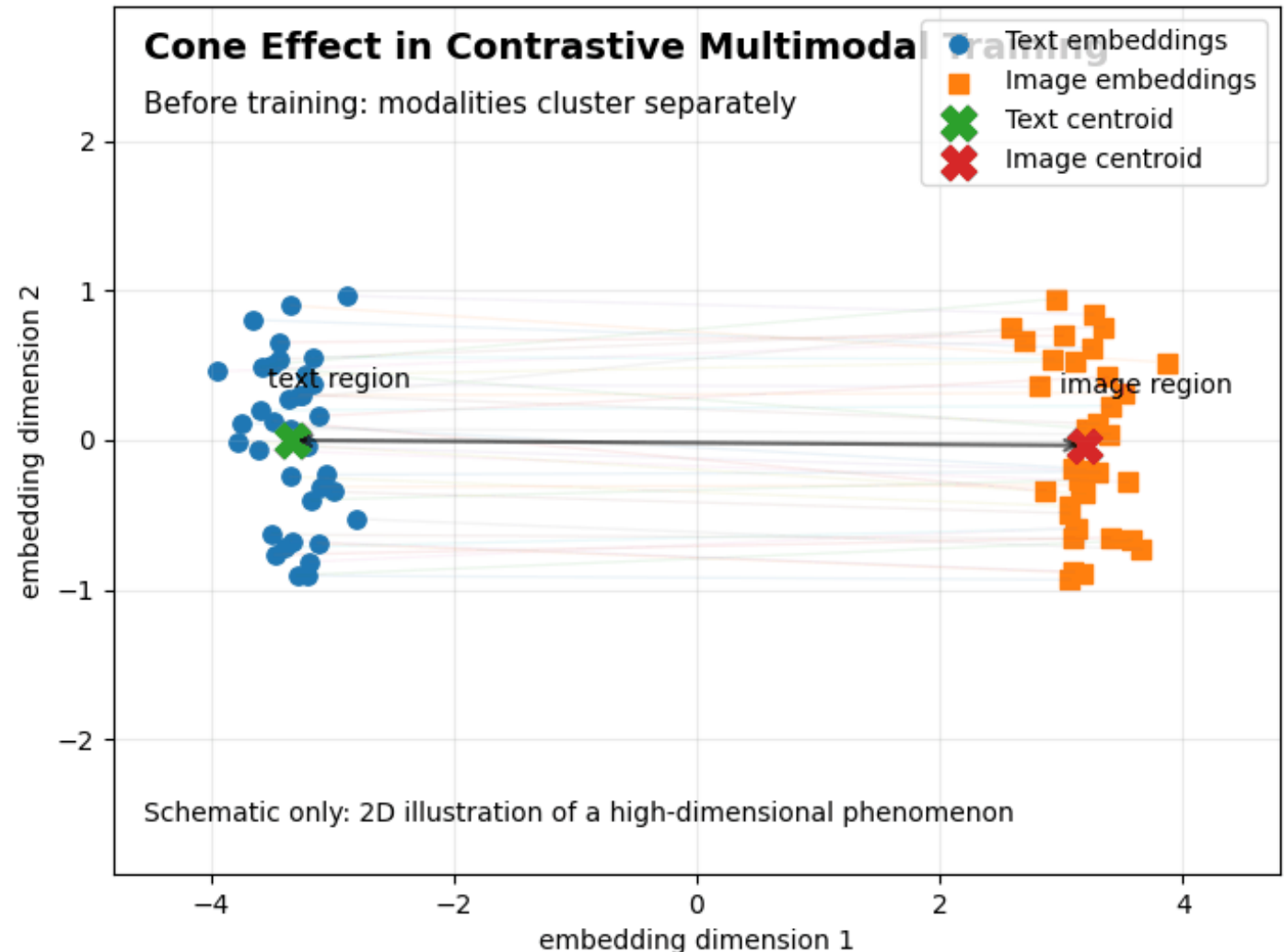
Table 4: Comparison of cross-modal alignment scores on 1K of random samples from each dataset.

Jina

Core Effect

- Contrastive multimodal training can create a bias known as the **cone effect**.
- Each modality initially forms its **own cluster** in embedding space.
- Training pulls matched cross-modal pairs together.
- But instead of fully mixing the modalities, the embeddings form a **high-dim** cone-like bridge.

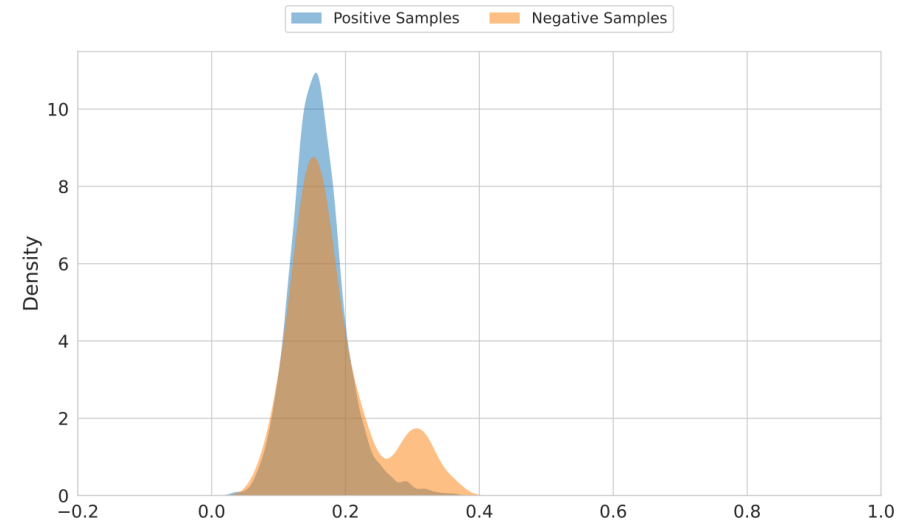
alignment happens through a structured connection, not a truly uniform shared space.



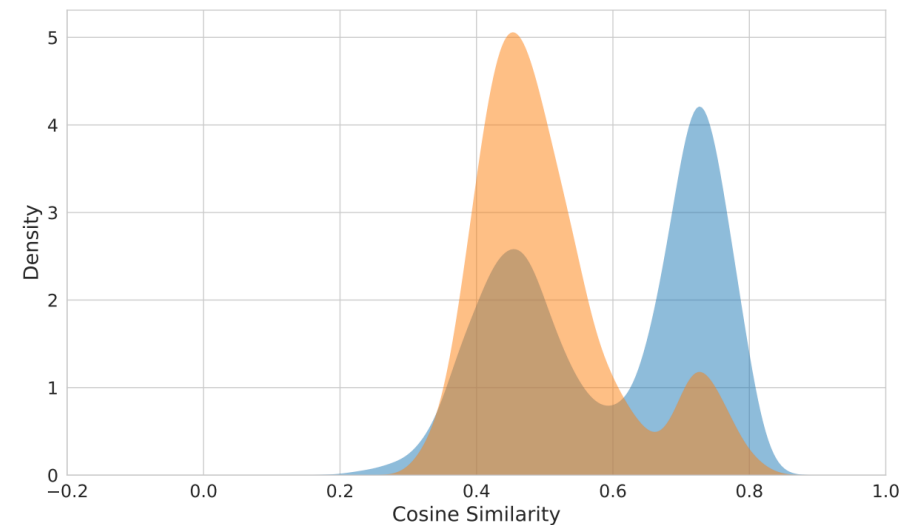
Jina

Core Effect

- Contrastive multimodal training can create a bias known as the **cone effect**.
- Each modality initially forms its **own cluster** in embedding space.
- Training pulls matched cross-modal pairs together.
- But instead of fully mixing the modalities, the embeddings form a **high-dim** cone-like bridge.



OpenAI CLIP

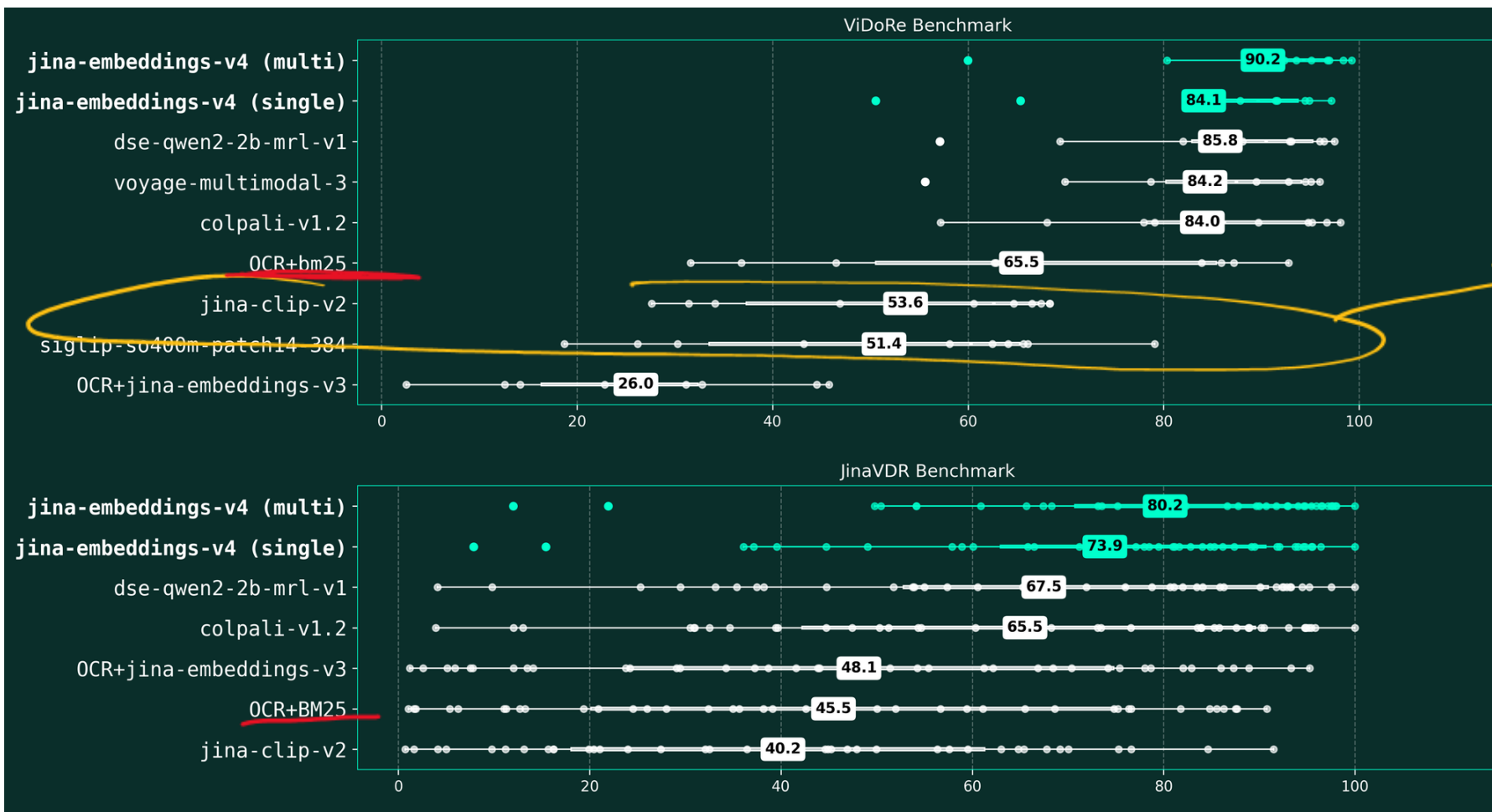


Jina-embedding-v4

Jina

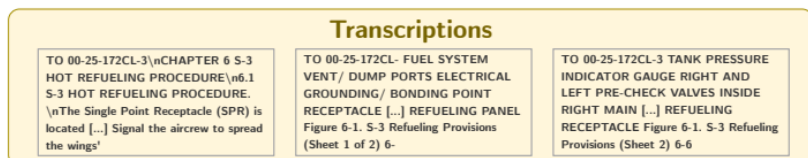
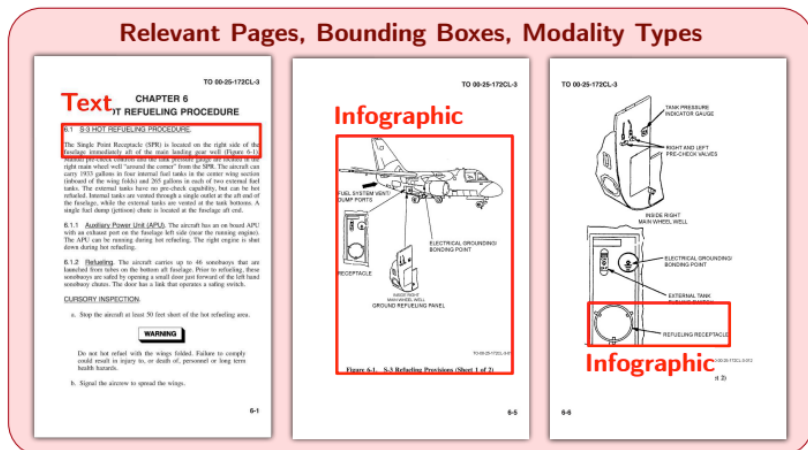
- OCR+bm2.5 generally better than CLIP-based for vis-doc retrieval
- ColPali > MRAG 1.0 (OCR-based) thanks to later interaction
- Jina's multi-vector > single-vector

MRAG 2.0 or 1.0?



Multimodal RAG Roadmap

Query: Where is the S-3's single point receptacle located?



Answer: The S-3's single point receptacle (SPR) is located on the right side of the fuselage immediately aft of the main landing gear well.

Figure 1: **ViDoRe V3 sample.** Each query is annotated with the relevant pages, a document-grounded answer, bounding boxes localizing supporting evidence and modality labels for each bounding box. Documents are provided in image, text and PDF formats.

- Visual retrievers outperforms textual ones; (even with txt query)
- Text rerankers outperforms visual ones

Model	English Datasets							French Datasets			Avg.
	C.S.	Nucl.	Fin.	Phar.	H.R.	Ind.	Tel.	Phys.	Ener.	Fin.	
<i>Textual pipeline</i>											
Jina-v4 ^{textual}	64.3	44.3	48.4	54.9	52.8	38.4	56.3	43.6	60.1	41.3	50.4
+ zerank-2	82.1 ↑17.8	53.5 ↑9.2	69.2 ↑20.8	66.2 ↑11.3	66.5 ↑13.7	53.2 ↑14.8	71.5 ↑15.2	48.2 ↑4.6	71.5 ↑11.4	53.7 ↑12.4	63.6 ↑13.2
<i>Visual pipeline</i>											
Jina-v4 ^{visual}	71.8	50.0	59.3	63.1	59.5	50.4	64.8	46.6	64.0	46.1	57.6
+ jina-reranker-m0	76.7 ↑4.9	50.8 ↑0.8	59.2 ↓0.1	65.4 ↑2.3	56.0 ↓3.5	50.9 ↑0.5	70.8 ↑6.0	46.9 ↑0.3	61.7 ↓2.3	39.8 ↓6.3	57.8 ↑0.2

Table 2: Retrieval performance (NDCG@10) of retriever + reranker pipelines.

- Hybrid retrieval performs the best (F1 scores)

Modality	English Datasets							French Datasets			Avg.
	C.S.	Nucl.	Fin.	Phar.	H.R.	Ind.	Tel.	Phys.	Ener.	Fin.	
Visual	39.4	25.5	28.4	27.5	30.0	21.4	31.4	26.6	25.2	22.9	27.8
Textual	35.4	23.1	24.5	24.2	27.4	16.5	29.0	25.8	23.9	20.4	25.0
Hybrid	43.0	27.7	30.9	29.7	32.6	22.2	35.5	26.5	29.8	24.3	30.2
Avg. # Pages for hybrid	6.96	7.38	7.77	7.40	7.29	7.77	7.09	7.26	6.97	7.61	7.35

Hybrid: concatenate the top-5 results from the visual retriever (images) with the top-5 results from the textual retriever (text), without removing duplicates

Multimodal RAG Roadmap

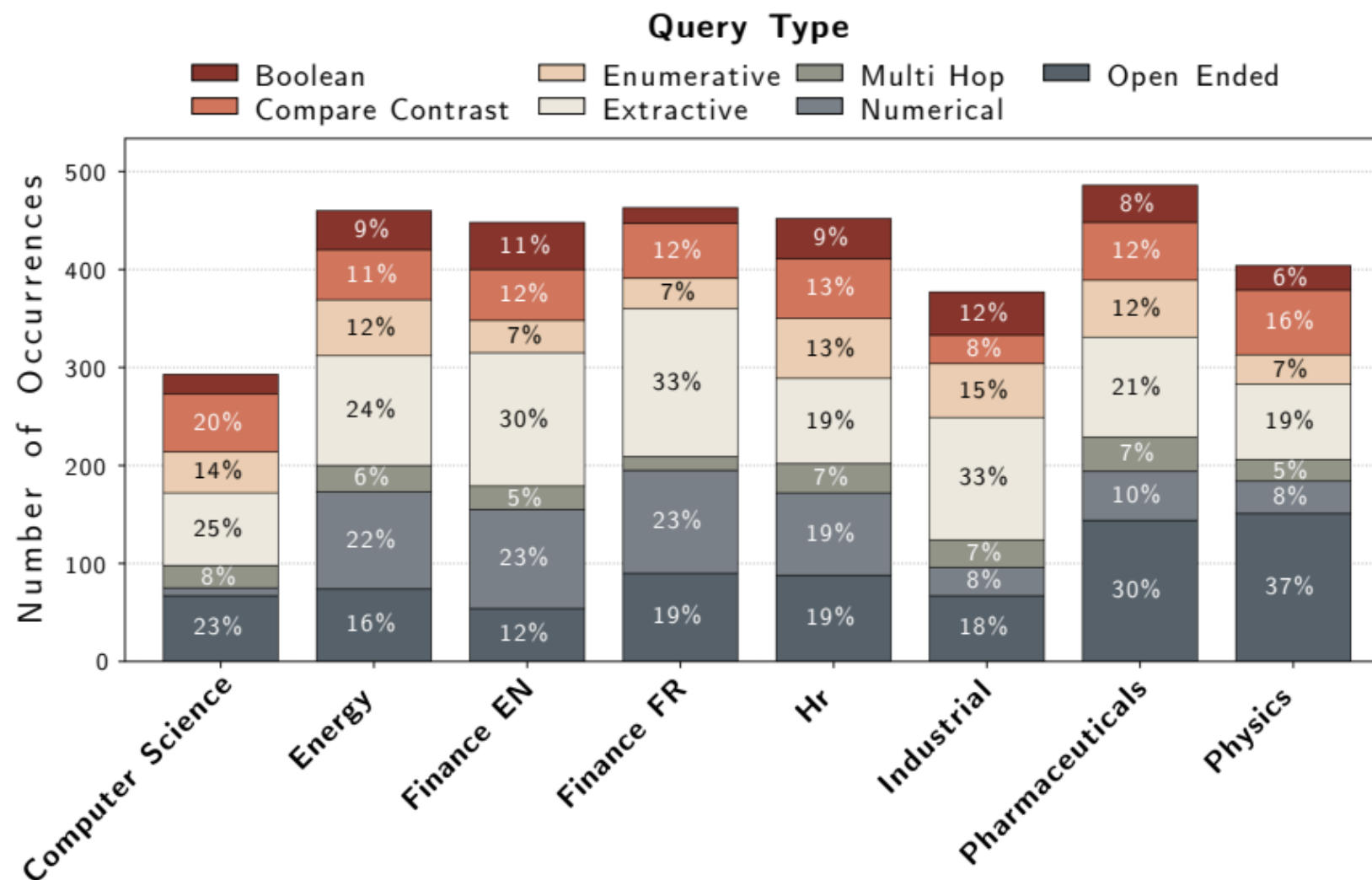
- **Image-based context outperforms text-based ones:** providing more visual cues for grounding

Retrieval pipeline	Context modality	Generation model	English Datasets					French Datasets			Avg. Hard	Avg. Easy	Avg. Global
			C.S.	Fin.	Phar.	H.R.	Ind.	Phys.	Ener.	Fin.			
Oracle	Text	Gemini 3 Pro	80.9	70.2	71.4	72.3	66.4	71.2	69.2	62.8	62.3	79.3	70.6
	Image		86.5	70.6	76.1	71.1	68.2	74.5	69.8	64.1	64.7	79.7	72.6
	Hybrid		86.0	68.9	73.4	70.4	65.4	69.2	69.5	62.8	63.4	77.5	70.7
Jina-v4 ^{text.} + zerank-2	Text	Gemini 3 Pro	80.9	66.0	59.9	63.2	60.4	69.2	64.9	<u>54.7</u>	52.1	75.5	64.9
Jina-v4 ^{text.} + zerank-2 & ColEmbed-3B-v2	Hybrid	Gemini 3 Pro	85.1	65.0	65.9	64.8	59.4	69.9	62.7	52.8	<u>54.7</u>	76.6	65.7
ColEmbed-3B-v2	Text	Gemini 3 Pro	82.3	62.5	61.0	62.9	56.2	64.9	62.3	49.4	51.7	73.0	62.7
		Kimi K2	81.4	56.6	59.1	55.7	55.8	73.8	60.4	43.1	44.6	74.3	60.7
	Image	Gemini 3 Pro	83.3	<u>67.3</u>	62.9	65.4	57.2	67.9	64.3	47.8	54.5	74.1	64.5
		Gemini 3 Flash	80.9	64.1	63.5	63.8	55.1	68.2	63.3	47.8	50.3	74.4	63.3
		GPT-5.2	86.5	59.5	<u>68.1</u>	<u>66.0</u>	<u>61.5</u>	76.5	<u>66.2</u>	49.1	54.1	<u>78.1</u>	<u>66.7</u>
		Qwen3-VL-235B	86.0	59.9	64.0	60.7	57.2	71.9	59.7	44.4	51.0	74.1	63.0

Table 3: **End-to-end evaluation of final answer generation.** We report the percentage of correct final answers as determined by an LLM judge across the 8 public datasets. "Oracle" rows represent the upper-bound performance using gold-standard contexts. Average Easy and Average Hard denote performance stratified by query difficulty. For each column, the best result is **bolded** and the best non-oracle result is underlined.

Multimodal RAG Roadmap

- Which type of questions you think it is more challenging for Multimodal RAG?



Multimodal RAG Roadmap

- Which type of questions you think it is more challenging for Multimodal RAG?

Domain: **Pharmaceuticals**

Query id: **17**

Query type: **Extractive**

Query format: **Question**

Query: According to FDA guidelines, what is the required format for displaying the strength of a small volume parenteral drug on its label?

Answer: According to FDA guidelines, the strength of a small-volume parenteral drug must be displayed as the quantity per total volume (e.g., 2 g/20 mL), immediately followed in parentheses by the quantity per milliliter (e.g., 100 mg/mL).



Multimodal RAG Roadmap

- Which type of questions you think it is more challenging for Multimodal RAG?

Domain: **Human Resources**

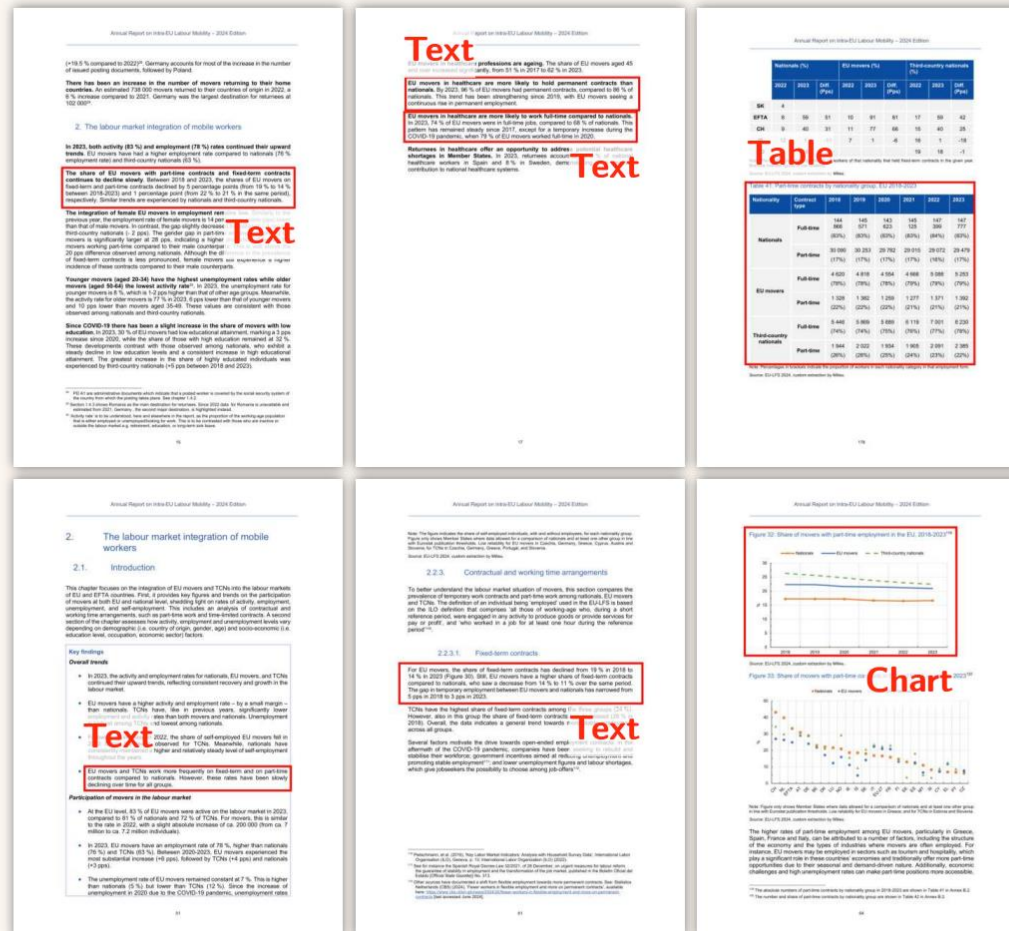
Query id: **0**

Query type: **Multi-hop**

Query format: **Question**

Query: What impact did the shifts in temporary contract usage from 2018 to 2023 have on the difference in employment stability between mobile EU workers and citizens of the host countries?

Answer: From 2018 to 2023, the job stability gap between mobile EU workers and nationals narrowed. The use of temporary contracts declined more for EU movers (from 19% to 14%) than for nationals (14% to 11%), reducing the difference between the two groups from 5 to 3 percentage points.



Multimodal RAG Roadmap

- Which type of questions you think it is more challenging for Multimodal RAG?

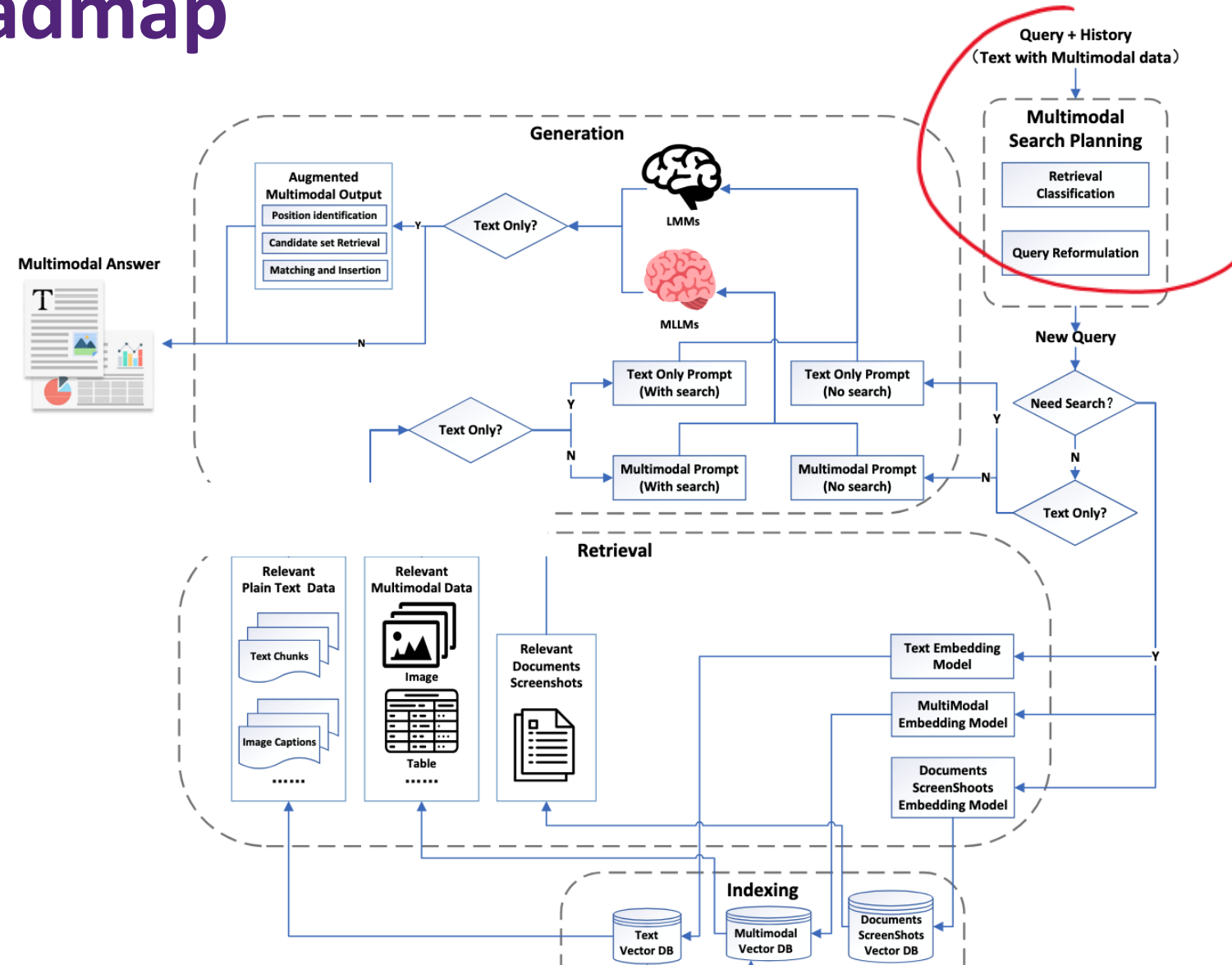
Category	Definition	Example
<i>Query Types</i>		
Open-ended	Seeks explanatory or descriptive information that requires synthesis.	What drives the rise in women's workforce involvement in EU nations?
Extractive	Requires the retrieval of a specific piece of information.	Bank of America preferred stock MM dividend rate
Compare Contrast	Mandates a comparison between multiple entities or data points.	Explain the factors contributing to the reduction in R2R rates for ANDAs.
Boolean	Poses a yes/no question necessitating multi-step reasoning.	Did JPMorganChase execute more than half of its planned repurchase program?
Numerical	Asks for a specific quantitative value that must be derived or calculated.	percentage increase in Morgan Stanley revenue from 2023 to 2024
Multi-hop	Requires integrating information from multiple sections or sources.	Summarize the steps involved in error reporting in ISMP's MERP.
Enumerative	Requests a list of all instances sharing a common property.	Specify the ISCO codes used to define domestic workers in the EU.
<i>Query Formats</i>		
Question	An interrogative sentence.	What was Citigroup's net interest margin in 2024?
Keyword	A non-verbal phrase or set of terms.	female employment rate European Union 2023
Instruction	A directive specifying a task.	Identify the use case of a drill point gauge.



Multimodal RAG Roadmap

MRAG 3.0:

- move toward **end-to-end** multimodality by keeping document screenshots/page-level representations,
- adding **multimodal search planning**, and even enabling **multimodal outputs** instead of text-only answers.



Multimodal RAG Roadmap

Scenario I

Multimodal Data is Answer

What do the Alaskan Malamute and the Siberian Husky look like, respectively? 



The left is the Alaskan Malamute, and the right is the Siberian Husky .



What was the route of Columbus's voyage? 



You can learn the route of Columbus's voyage through the video below.



Scenario II

Multimodal Data Enhances Accuracy

How to make steamed egg custard? 



1: Place the eggs and a small amount of salt into a container, and mix well.



2: Add warm water and continue to stir, then strain it.



3: Steam for 10 minutes, drizzle with sesame oil. It's done!



Watch the video below for more detailed instructions.



Scenario III

Multimodal Data Enhances Richness

Please introduce the Egyptian pyramids. 

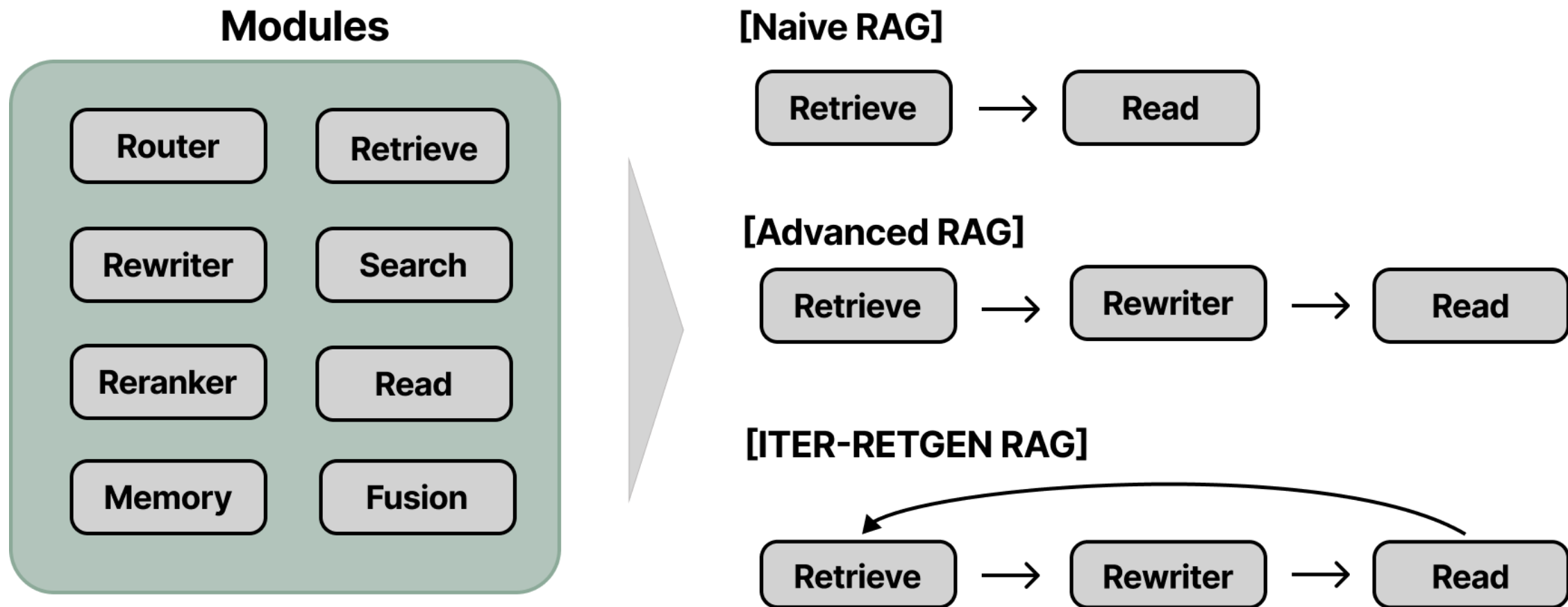


The Egyptian pyramids are among the most iconic and fascinating structures in history. Built primarily as tombs for the pharaohs of ancient Egypt, they served as grand final resting places to ensure their safe passage into the afterlife. The pyramids are mainly located in Giza, near Cairo. The Great Pyramid of Khufu is the largest and most well-known, standing at an astonishing height.



Constructed with precision, these pyramids were built using massive limestone blocks. Thousands of workers, including skilled artisans, were involved in their construction, which took many years. The architecture is remarkable, with the sides of the pyramids aligned almost perfectly with the four cardinal directions.

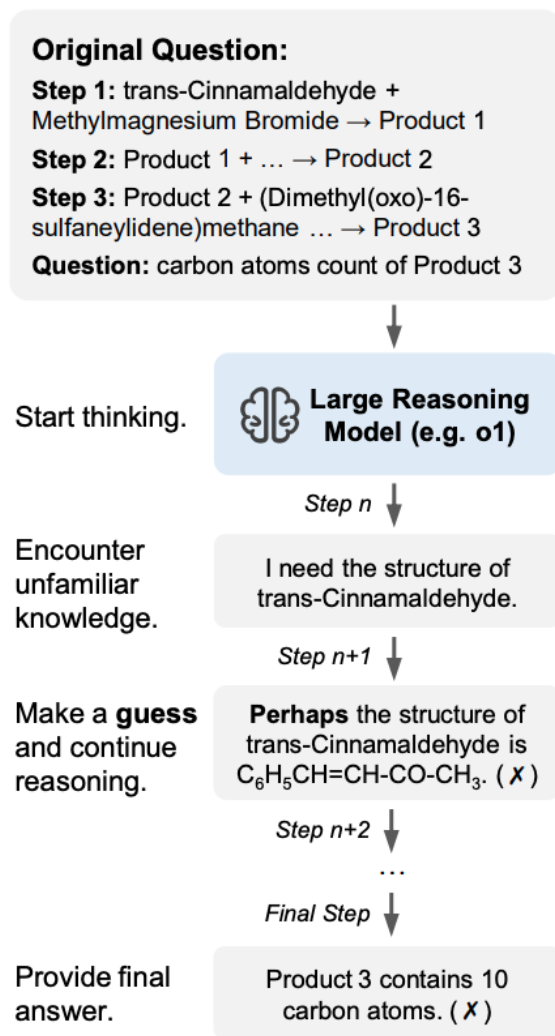
Multimodal RAG Roadmap



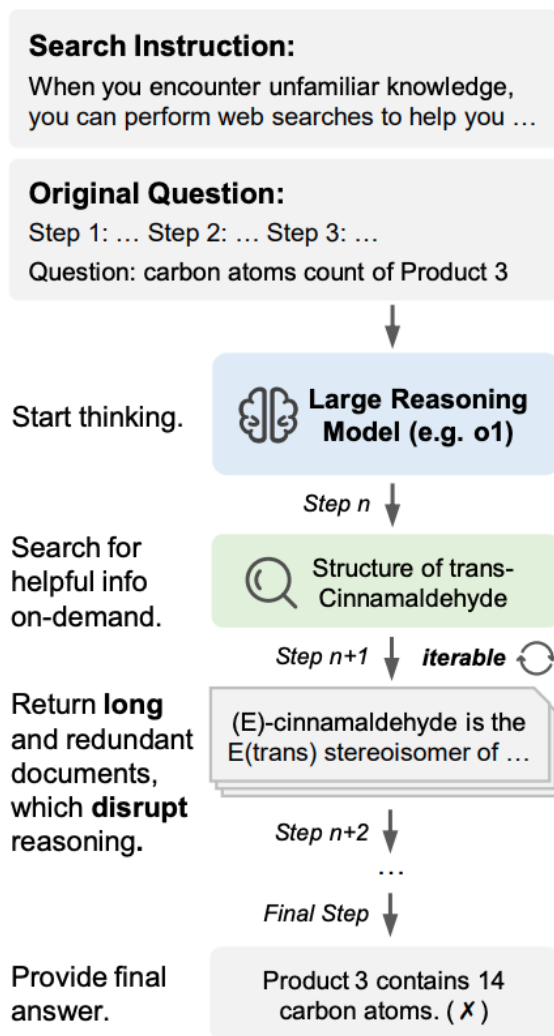
- **Agentic RAG (e.g., DeepResearch):** decide which tool to use -> check if re-retrieval is needed

Agentic RAG

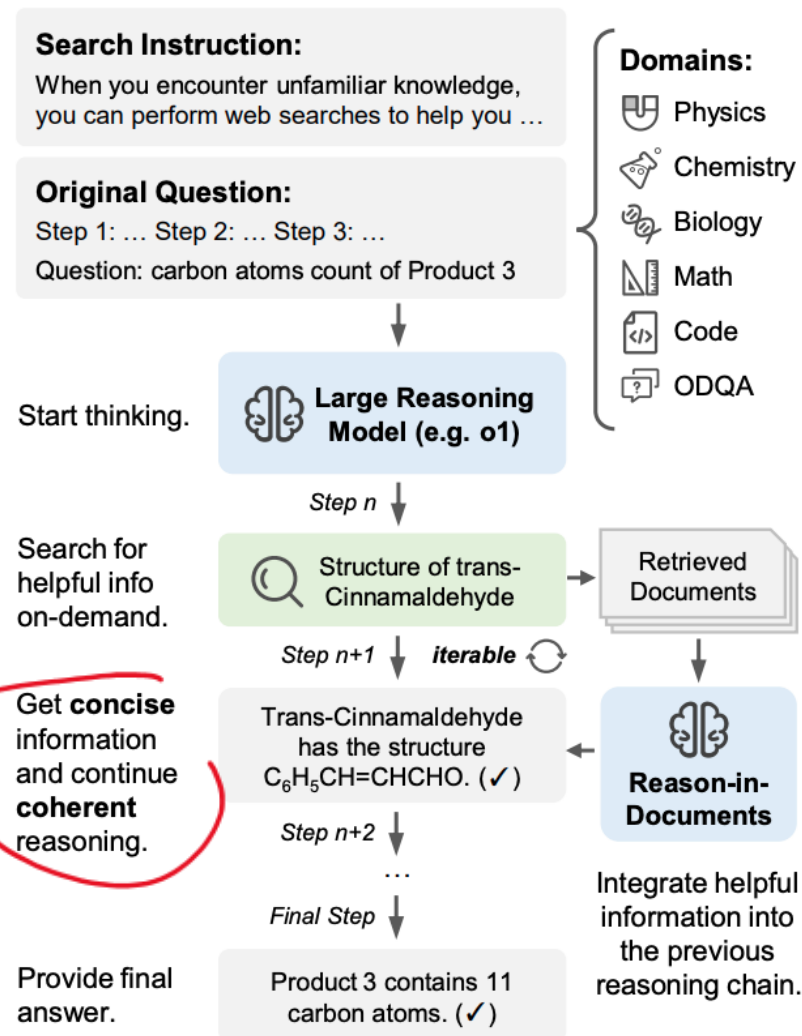
(a) Vanilla Reasoning Pattern



(b) Reason with Agentic RAG (Ours)

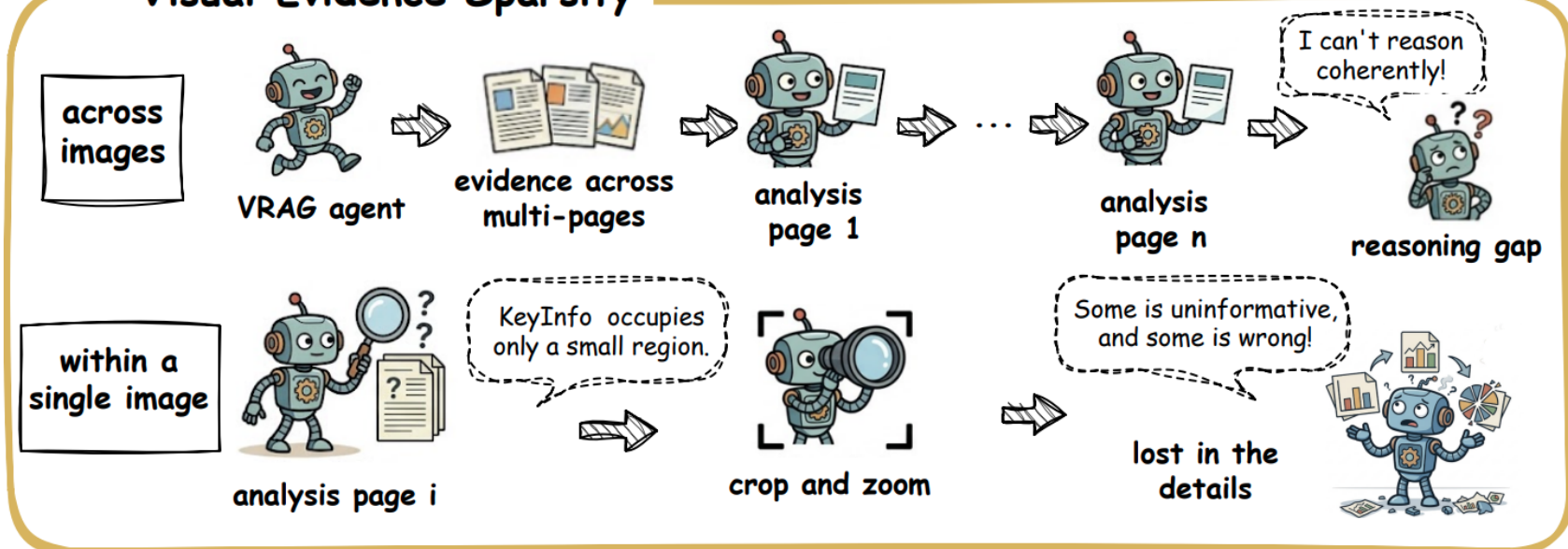


(c) The Search-o1 Framework (Ours)



Agentic RAG

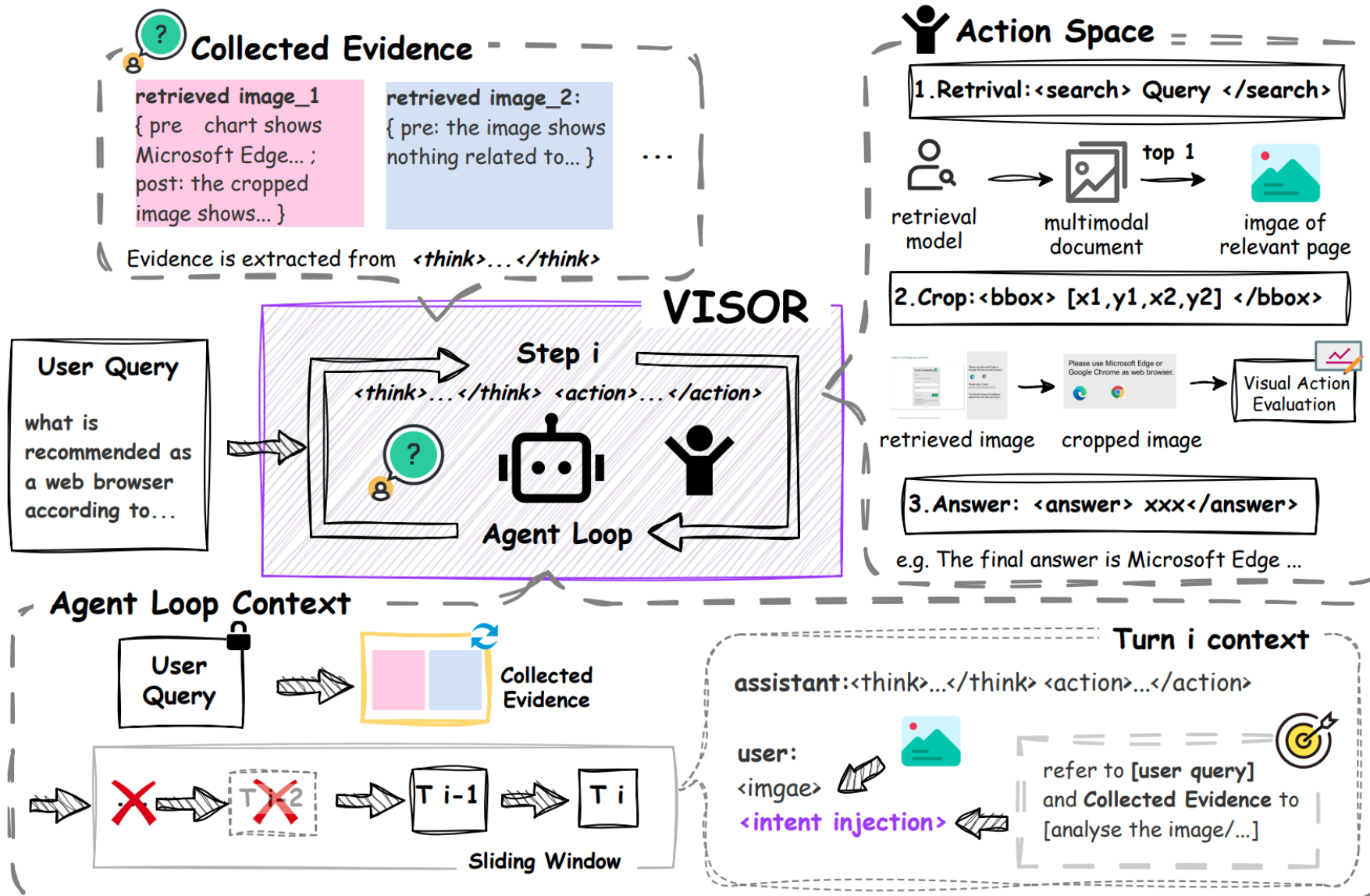
Visual Evidence Sparsity



Search Drift in Long-Horizon Interaction

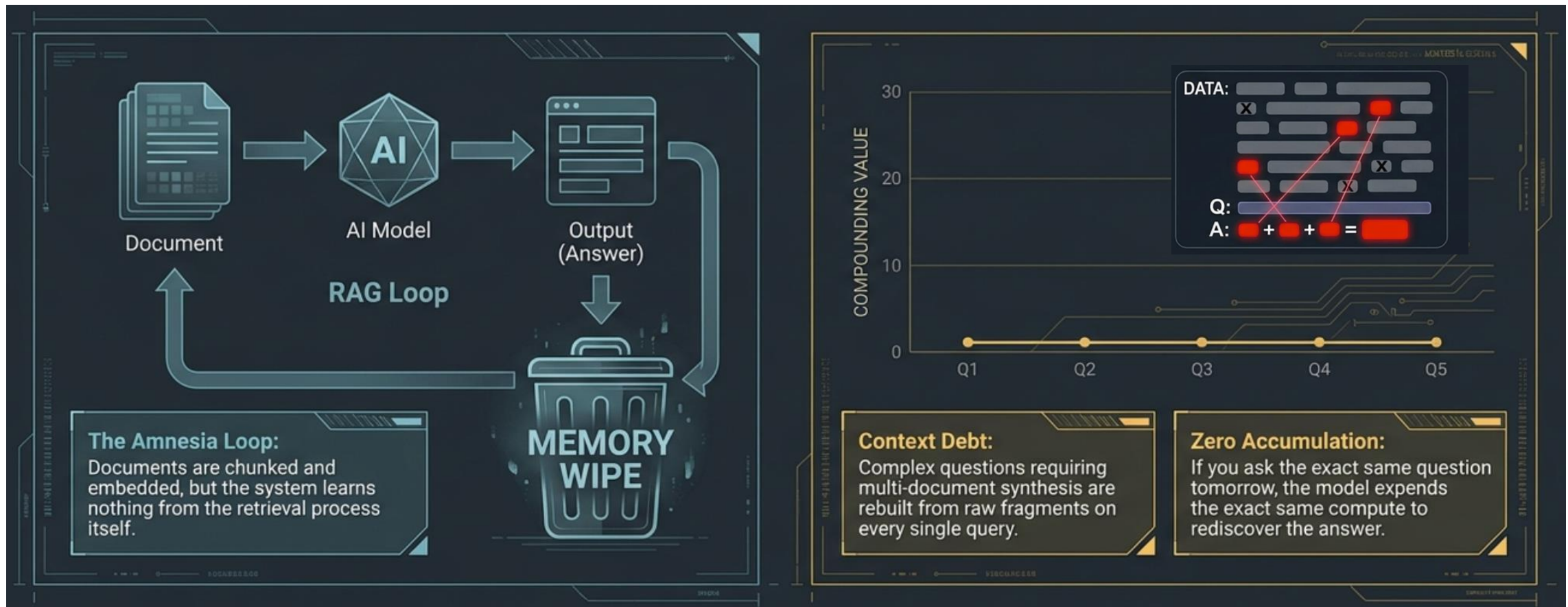


Agentic RAG



Is RAG always needed?

- RAG pipeline treats knowledge as a transient commodity, sometimes *cognitive load*



Is RAG always needed?

llm-wiki.md Raw

LLM Wiki

A pattern for building personal knowledge bases using LLMs.

This is an idea file, it is designed to be copy pasted to your own LLM Agent (e.g. OpenAI Codex, Claude Code, OpenCode / Pi, or etc.). Its goal is to communicate the high level idea, but your agent will build out the specifics in collaboration with you.

The core idea

Most people's experience with LLMs and documents looks like RAG: you upload a collection of files, the LLM retrieves relevant chunks at query time, and generates an answer. This works, but the LLM is rediscovering knowledge from scratch on every question. There's no accumulation. Ask a subtle question that requires synthesizing five documents, and the LLM has to find and piece together the relevant fragments every time. Nothing is built up. NotebookLM, ChatGPT file uploads, and most RAG systems work this way.

The idea here is different. Instead of just retrieving from raw documents at query time, the LLM incrementally builds and maintains a persistent wiki — a structured, interlinked collection of markdown files that sits between you and the raw sources. When you add a new source, the LLM doesn't just index it for later retrieval. It reads it, extracts the key information, and integrates it into the existing wiki — updating entity pages, revising topic summaries, noting where new data contradicts old claims, strengthening or challenging the evolving synthesis. The knowledge is compiled once and then *kept current*, not re-derived on every query.

This is the key difference: the wiki is a persistent, compounding artifact. The cross-references are already there. The contradictions have already been flagged. The synthesis already reflects everything you've read. The wiki keeps getting richer with every source you add and every question you ask.

You never (or rarely) write the wiki yourself — the LLM writes and maintains all of it. You're in charge of sourcing, exploration, and asking the right questions. The LLM does all the grunt work — the summarizing, cross-referencing, filing, and bookkeeping

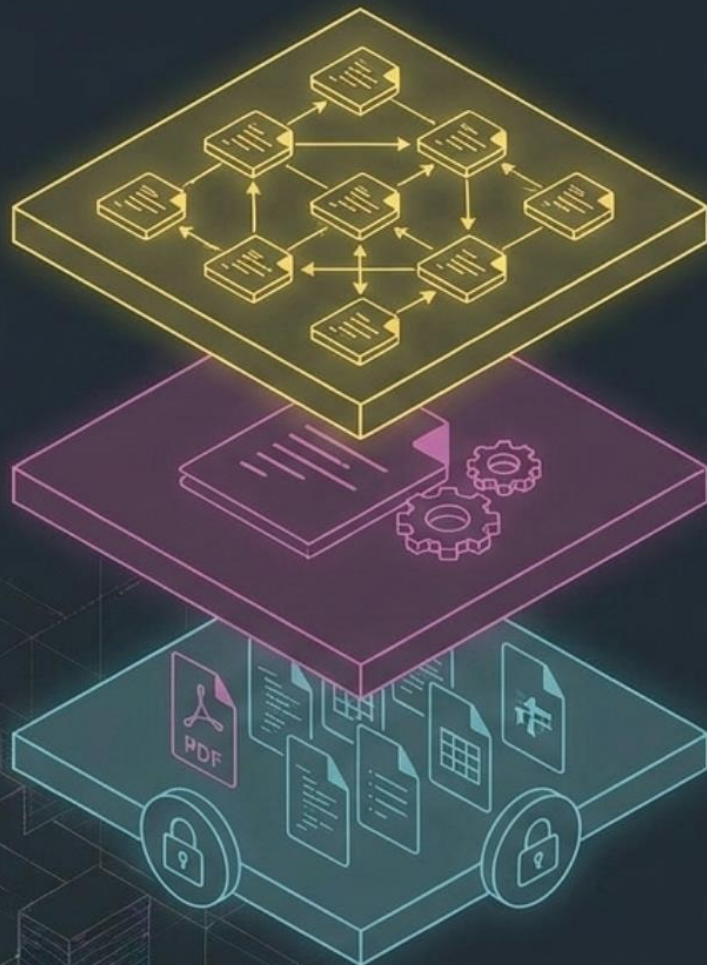
“What you need is LLM Wiki!”



Andrej Karpathy (Founding member at OpenAI)

What is LLM Wiki and How does it work?

The three-layered memory architecture



03. The Wiki (Compiled State)

The directory of LLM-generated markdown files. The agent owns this layer entirely, maintaining cross-references, topic summaries, and entity pages.

02. The Schema (Governance)

A single configuration file (e.g., `AGENTS.md`). This is the contract between human intent and AI execution. It defines entity types, naming conventions, and workflows.

01. Raw Sources (Immutable)

Your curated collection of PDFs, transcripts, and datasets. The LLM reads from this directory but is strictly prohibited from modifying it. This is auditable ground truth.

What is LLM Wiki and How does it work?

Navigating the compiled state

Two essential files prevent the agent from getting lost in its own creation.

3 operations

index.md (The spatial map)

```
+ entities/  
|   ├── company_A.md  
|   └── project_X.md  
+ concepts/  
|   ├── architecture.md  
|   └── workflow.md
```

A catalog of every page, categorized by entities and concepts, featuring one-line summaries. Before answering a query, the LLM reads the **index** to **pinpoint relevant files**, avoiding brute-force search.

log.md (The chronological ledger)

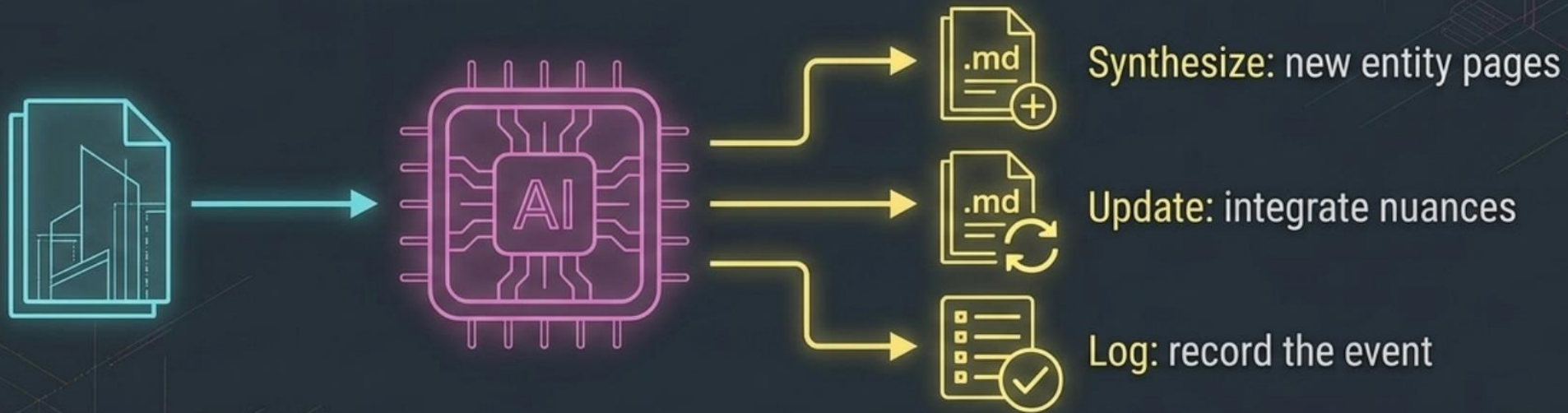
```
> grep "^# \[" log.md | tail -5  
## [2023-10-27 14:30:01] INGEST: Loaded 3 source documents.  
## [2023-10-27 14:35:45] QUERY: Answered user request on  
architecture.  
## [2023-10-27 14:40:12] MAINTENANCE: Updated cross-references  
for 'project_X'.  
## [2023-10-27 14:45:55] QUERY: Generated summary for  
'company_A'.  
## [2023-10-27 14:50:30] INGEST: Processed new transcript file.
```

An append-only record of every ingest, query, and maintenance pass. Provides the LLM with a **timeline** of the wiki's evolution and **temporal priority** when new data contradicts old claims.

What is LLM Wiki and How does it work?

Operation 01: Ingestion and Compilation

Adding a new document triggers a system-wide compilation pass.



Extract

The LLM reads the raw source and identifies novel concepts versus established facts.

Update

It seamlessly integrates new nuances into existing topic pages without destroying historical context.

Synthesize

It creates new entity pages for entirely new concepts, generating `[[wikilinks]]` to tie them in.

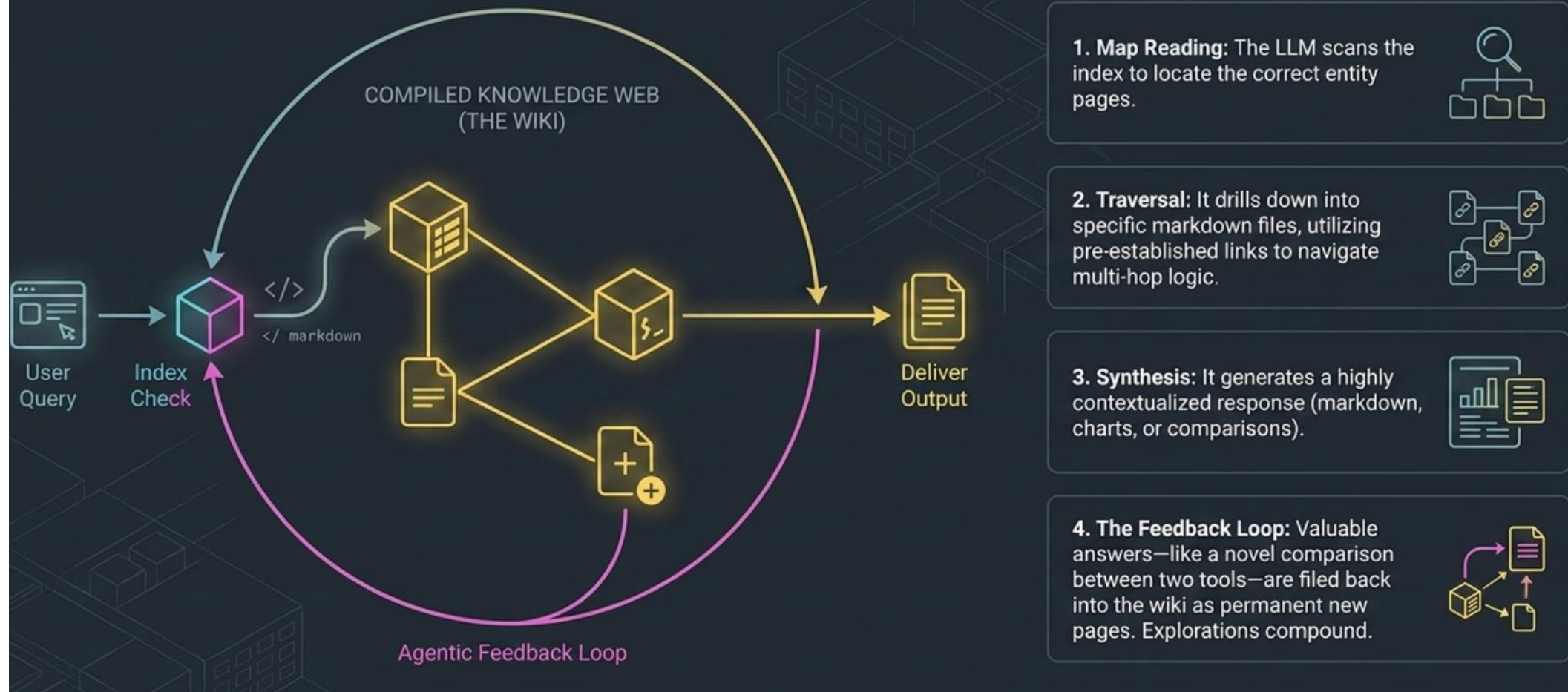
Log

It records the compilation event for future audibility.

What is LLM Wiki and How does it work?

Operation 02: Query and Traversal

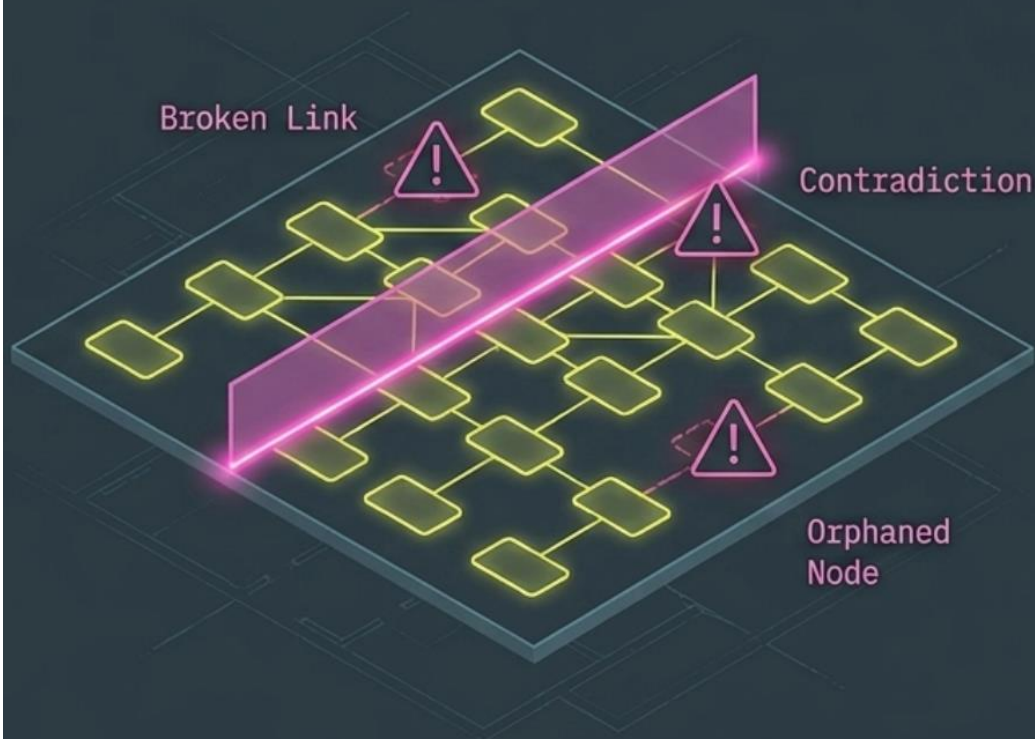
Answers are synthesized from pre-organized knowledge, not raw fragments.



What is LLM Wiki and How does it work?

Operation 03: Linting and Self-Healing

Because unmonitored LLMs drift, the wiki requires routine autonomic maintenance.



Detect Contradictions

Flags instances where a newer raw source supersedes an older claim, prompting a resolution.



Prune Orphans

Identifies pages with no inbound links, ensuring the knowledge graph remains fully traversable.



Repair Infrastructure

Automatically fixes broken `[[wikilinks]]` resulting from page renames or merges.



Identify Gaps

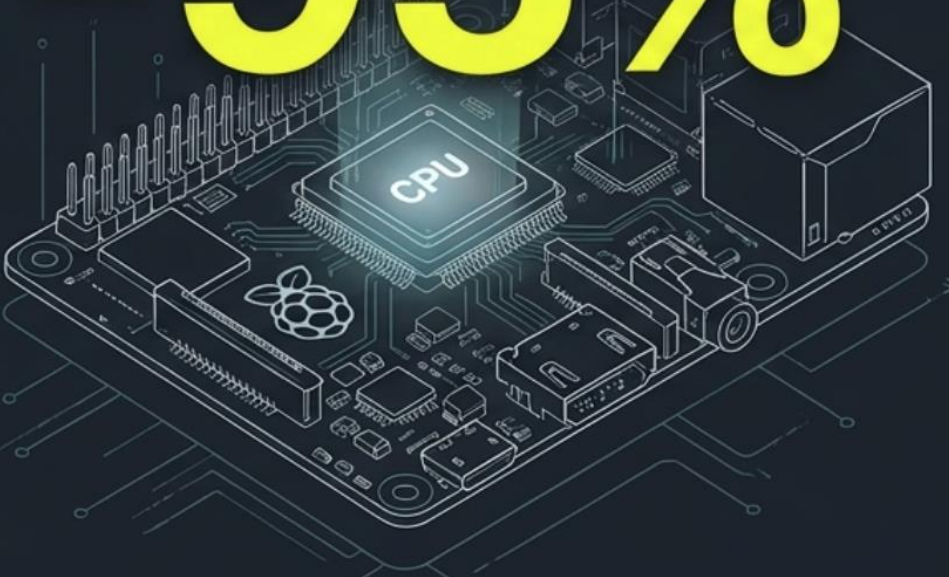
Proposes missing concepts that require external web searches to complete the cognitive map.



What is LLM Wiki and How does it work?

Token economics and offline edge deployment

Pre-synthesizing knowledge slashes inference costs and unlocks privacy.



-95%

95% Token Reduction: By loading a concise, curated markdown file into the context window instead of chunked, raw PDFs, token consumption drops drastically for small-to-medium knowledge bases.

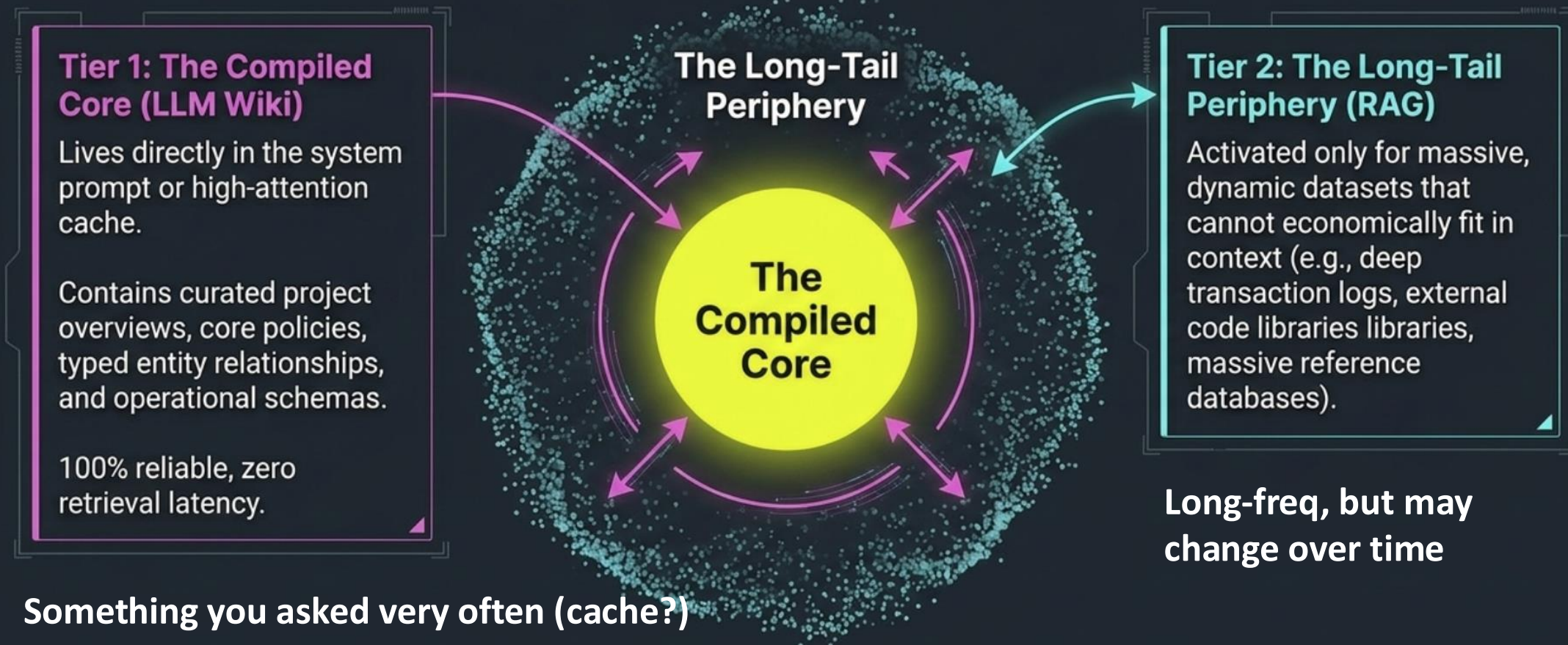
Air-Gapped Viability: Because the heavy lifting is handled by simple file I/O operations and markdown, the entire wiki can run locally.

Edge Hardware Ready: Efficient enough to run on an M2 MacBook with Ollama, or deployed fully offline on a Raspberry Pi 5 with a quantized sub-8B model. Zero data leaves the box.

What is LLM Wiki and How does it work?

The Hybrid Future: Hierarchical Memory

The optimal architecture combines synthesis with retrieval.



Is RAG always needed?

Graphify is a concrete example of the compiled-memory idea

 **graphify** Public

 Sponsor

 Watch 128

 Fork 3.4k

 Starred 31k

 v4

 5 Branches

 55 Tags

 Add file

 Code

 safishamsi	readme: revert to pepy.tech total downloads badge ✓	7a0a5ac · 2 days ago	🕒 169 Commits
📁 .github	Add v4 to CI branch triggers		last week
📁 graphify	fix #432 to_html crash on large graphs, fix #431 Go import...		2 days ago
📁 tests	v0.4.15: VS Code Copilot Chat, OpenCode/Gemini Window...		4 days ago
📁 worked	update worked example READMEs for v3 multi-platform		2 weeks ago
📄 .gitignore	feat: add Trae and Trae CN platform support		2 weeks ago
📄 AGENTS.md	Update CHANGELOG for 0.4.14, fix AGENTS.md rebuild co...		5 days ago
📄 ARCHITECTURE.md	docs: update surprising connections description test count		2 weeks ago
📄 CHANGELOG.md			

Andrej Karpathy keeps a `/raw` folder where he drops papers, tweets, screenshots, and notes. graphify is the answer to that problem - 71.5x fewer tokens per query vs reading the raw files, persistent across sessions, honest about what it found vs guessed.

```
/graphify .
```

```
# works on any folder - your codebase, notes, papers, anyt
```

About

AI coding assistant skill (Claude Code, Codex, OpenCode, Cursor, Gemini CLI, GitHub Copilot CLI, OpenClaw, Factory Droid, Trae, Google Antigravity). Turn any folder of code, docs, papers, images, or videos into a queryable knowledge graph

safishamsi.github.io/penpax.ai

skills gemini knowledge-graph
codex graphrag antigravity
claude-code openclaw

What we covered today...

Take-home Message

- **Naïve RAG is brittle:** chunking, retrieval, and ranking all matter.
- **Better RAG is a pipeline design problem:** hybrid search, reranking, and query planning improve evidence quality.
- **Multimodal RAG is not text RAG with images attached:** preserving original visual evidence matters.
- **Not all knowledge should be retrieved online:** some can be compiled into LLM Wiki / graph memory.
- **Future systems will be hybrid:** retrieve fresh/raw evidence, compile stable/reusable knowledge.

Next Week: Evaluation of Multimodal RAG



THE UNIVERSITY
OF QUEENSLAND
AUSTRALIA

CREATE CHANGE

THANKS!

Do you have any questions?

uqyluo@uq.edu.au

The University of Queensland

